

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-104378-115323

**Mobilná aplikácia na ovládanie výrobnéj linky s
využitím jedinečného identifikátora**

Diplomová práca

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-104378-115323

Mobilná aplikácia na ovládanie výrobnéj linky s využitím jedinečného identifikátora

Diplomová práca

Študijný program:	Aplikovaná mechatronika a elektromobilita
Študijný odbor:	kybernetika
Školiace pracovisko:	Ústav automobilovej mechatroniky (FEI)
Školiteľ:	Ing. Lukáš Beňo PhD.



ZADANIE BAKALÁRSKEJ PRÁCE

Študent: **Meno Priezvisko**
ID študenta: 012345
Študijný program: jadrové a fyzikálne inžinierstvo
Študijný odbor: elektrotechnika
Vedúci práce: tituly, Meno Priezvisko, tituly
Vedúci pracoviska: tituly Meno Priezvisko, tituly
Miesto vypracovania: Ústav jadrového a fyzikálneho inžinierstva

Názov práce: **Rozšírená šablóna záverečnej práce na FEI STU v Bratislave
v systéme LaTeX**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

- Vytvorte šablónu záverečnej práce pre študentov bakalárskeho a inžinierskeho štúdia na FEI STU v Bratislave.
- Napište návod na písanie záverečnej práce s použitím vytvorenej šablóny.
- Vytvorte prehľad najpoužívanejších typov citovaných zdrojov a zdokumentujte ich tak, aby boli v súlade s normou ISO 690.

Zoznam odbornej literatúry:

- Zákon č. 131/2002 Z. z. z 21. februára 2002 o vysokých školách a o zmene a doplnení niektorých zákonov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2002/131/ZZ_2002_131_20240901.pdf
- Vyhláška č. 233/2011 Z. z. Ministerstva školstva, vedy, výskumu a športu Slovenskej republiky z 1. júla 2011, ktorou sa vykonávajú niektoré ustanovenia zákona č. 131/2002 Z. z. o vysokých školách a o zmene a doplnení niektorých zákonov v znení neskorších predpisov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2011/233/ZZ_2011_233_20201015.pdf

3. Metodické usmernenie Ministerstva školstva, vedy, výskumu a športu SR č. 56/2011 o náležitostiach záverečných prác, ich bibliografickej registrácii, kontrole originality, uchovávaní a sprístupňovaní č. 2011-11513/31015:4-071 [online]. [cit. 2024-09-24]. Dostupné na <http://www.minedu.sk/metodicke-usmernenie-c-562011-o-nalezitostiach-zaverecných-prac-ich-bibliografickej-registrácii-uchovavani-a-sprístupnovani/>.
4. STN ISO 214: 1998. Dokumentácia. Abstrakty (referáty) pre publikácie a dokumentáciu. Bratislava: Slovenský ústav technickej normalizácie. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
5. STN ISO 690: 2022. Dokumentácia – Bibliografické odkazy – Obsah, forma a štruktúra. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
6. STN 01 6910: 2023. Pravidlá písania a úpravy písomností. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
7. STN ISO 2145: 1997. Dokumentácia. Číslovanie oddielov a pododdielov písaných dokumentov. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
8. STN ISO EN 80 000-1: 2022. Veličiny a jednotky. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
9. LICHNEROVÁ, L. a B. BELLÉROVÁ. Nové pravidlá citovania podľa ISO 690 z roku 2021. *ITlib. Informačné technológie a knižnice*, 3–4/2023, 10 – 22. [cit. 2024-09-06]. Dostupné na <http://doi.org/10.52036/1335793X.2023.3-4.10-22>.
10. HOFTICH, M. ISO 690 biblatex style [online]. [cit. 2024-09-30]. Dostupné na <https://mirrors.ibiblio.org/CTAN/macros/latex/contrib/biblatex-contrib/biblatex-iso690/biblatex-iso690.pdf>.

Termín odovzdania bakalárskej práce: 31. 05. 2025

Dátum schválenia zadania bakalárskej práce: 28. 02. 2025

Zadanie bakalárskej práce schválil: tituly Meno Priezvisko, tituly – garant študijného programu

Podakovanie

Na tomto mieste by som sa rád poďakoval Ing. Lukášovi Beňovi PhD. za cenné rady a pomoc pri písaní diplomovej práce.

Abstrakt

Príručka pre študentov Fakulty elektrotechniky a informatiky STU v Bratislave obsahuje rady a návody, ako z formálneho hľadiska pristupovať k vypracovaniu záverečnej práce vysokoškolského štúdia. Dokument zároveň môže slúžiť ako šablóna práce v systéme na sadzbu textu $\text{\LaTeX}$. Detailne sa zaoberá pravidlami sadzby matematických rovníc, číslovania plávajúcich objektov a odkazovaním na ne. Podrobne sa venuje aj spôsobu citovania externých literárnych zdrojov.

Kľúčové slová

záverečná práca, šablóna, $\text{\LaTeX}$, formátovanie textu, citácie

Abstract

The manual for students of the Faculty of Electrical Engineering and Information Technology at Slovak University of Technology in Bratislava provides advice and guidelines on how to approach the formal aspects of writing a final thesis for university studies. The document can also serve as a template for the thesis in the $\text{\LaTeX}$ typesetting system. It covers in detail the rules for typesetting mathematical equations, numbering floating objects, and referencing them. It also thoroughly addresses the method of citing external literary sources.

Keywords

Final thesis, template, $\text{\LaTeX}$, text formatting, citations

Obsah

Úvod	1
1 Analýza existujúcich riešení	3
1.1 Siemens WinCC Unified	3
1.2 Ignition Perspective	4
1.3 Tulip	4
1.4 AVEVA InTouch Edge HMI	5
2 Prehľad technológií pre vývoj aplikácií a priemyselnú komunikáciu	8
2.1 Frontendové technológie	8
2.1.1 Natívny vývoj	8
2.1.2 React Native	9
2.1.3 Flutter	9
2.1.4 Webové aplikácie	10
2.2 Backendové technológie	11
2.2.1 Node.js a NestJS	11
2.2.2 Python a FastAPI	11
2.2.3 Go	12
2.3 Kontajnerizácia a Edge architektúra	12
2.3.1 Docker a Docker Compose	12
2.4 Priemyselné komunikačné protokoly	13
2.4.1 OPC UA	13
2.4.2 S7 Protokol	13
2.4.3 MQTT	13
2.4.4 Modbus TCP	14
3 Prehľad technológií pre inteligentných asistentov	15
3.1 Veľké jazykové modely - LLM	15
3.1.1 Definícia a princíp fungovania	15
3.1.2 Kľúčové parametre a vlastnosti	16
3.1.3 Otvorené vs. proprietárne modely	16
3.1.4 Platforma Ollama	17
3.2 Virtuálni AI asistenti	17
3.2.1 Definícia a klasifikácia	17
3.2.2 Architektúra konverzačného AI asistenta	18
3.2.3 Systémový prompt a kontrola správania	18

3.2.4	Výzvy a obmedzenia v priemyselnom prostredí	19
3.3	RAG	19
3.3.1	Motivácia a princíp	19
3.3.2	Architektúra RAG pipeline	20
3.3.3	Vektorové databázy a embedding modely	21
3.3.4	Framework LangChain	22
3.3.5	Výhody a obmedzenia RAG	22
4	Typy identifikátorov a ich použitie	24
4.1	QR kódy	24
4.2	Fiduciálne markery - AprilTag, ArUco, STag	24
4.3	3D identifikátory	24
4.3.1	Princíp a štruktúra	25
5	Špecifikácia požiadaviek systému	26
5.1	Prehľad systému	26
5.2	Popis používateľských rolí	27
5.3	Use-case scenáre	27
5.4	Funkcionálne požiadavky	29
5.5	Nefunkcionálne požiadavky	30
5.6	Doménové požiadavky	31
6	Voľba technológií	32
6.1	Komunikácia s PLC – S7 protokol	32
6.2	Komunikačná middleware vrstva – Node-RED	32
6.3	Transportný protokol – MQTT namiesto OPC UA	33
6.4	Backendový framework – FastAPI	33
6.5	Mobilný frontend – Flutter	34
6.6	TODO: Identifikátor - ArUco	34
7	Experimentálne porovnanie a výber jazykových modelov	35
7.1	Popis testovaných modelov	35
7.2	Metodika testovania	36
7.3	Výsledky porovnania	37
7.4	Analýza výsledkov a výber modelu	38
7.4.1	Overenie na cloudovom hardvéri	39
7.4.2	Záverečné rozhodnutie o architektúre AI asistenta	39
8	TOOD: Aplikácia a jej funkcionality	40

8.1	Skener Aruco kodov	40
8.2	Generátor Aruco kódov	40
8.3	Monitoring a ovládanie zariadení	40
8.4	Virtuálny asistent PLEXO	40
8.5	Správa používateľov	40
8.6	Správa zariadení	40
8.7	Správa Knowledgebase	41
8.8	Nastavenia	41
Záver		42
Literatúra		44
	Použitie nástrojov umelej inteligencie	44
A Algoritmus		45
B Výpis dlhého kódu		46
C Slovníček pojmov		50

Zoznam značiek a skratiek

AI	Artificial Intelligence (slovensky „umelá inteligencia“)
ANN	Approximate Nearest Neighbor (slovensky „vyhľadávanie aproximovaných najbližších susedov“)
API	Application Programming Interface (slovensky „aplikačné programové rozhranie“)
ASGI	Asynchronous Server Gateway Interface (slovensky „asynchrónne rozhranie serverovej brány“)
CLI	Command Line Interface (slovensky „rozhranie príkazového riadku“)
CPU	Central Processing Unit (slovensky „centrálna procesorová jednotka“)
GPU	Graphics Processing Unit (slovensky „grafická procesorová jednotka“)
HMI	Human-Machine Interface (slovensky „rozhranie človek-stroj“)
HNSW	Hierarchical Navigable Small World (slovensky „hierarchicky navigovateľné malé svety“)
HTML	HyperText Markup Language (slovensky „hypertextový značkovací jazyk“)
HTTP	HyperText Transfer Protocol (slovensky „hypertextový prenosový protokol“)
IIoT	Industrial Internet of Things (slovensky „priemyselný internet vecí“)
IoT	Internet of Things (slovensky „internet vecí“)
IT	Information Technology (slovensky „informačné technológie“)
IVF	Inverted File Index (slovensky „index invertovaných súborov“)
JSON	JavaScript Object Notation (slovensky „JavaScriptová objektová notácia“)
KB	Knowledge Base (slovensky „znalostná báza“)

LLM	Large Language Model (slovensky „veľký jazykový model“)
MQTT	Message Queuing Telemetry Transport (slovensky „protokol na prenos telemetrických správ“)
OPC UA	Open Platform Communications Unified Architecture (slovensky „zjednotená architektúra otvorenej platformovej komunikácie“)
OT	Operational Technology (slovensky „prevádzkové technológie“)
PLC	Programmable Logic Controller (slovensky „programovateľný logický automat“)
QoS	Quality of Service (slovensky „kvalita služieb“)
QR	Quick Response (slovensky „rýchla odozva“)
RAG	Retrieval-Augmented Generation (slovensky „generovanie rozšírené o vyhľadávanie“)
RAM	Random Access Memory (slovensky „pamäť s náhodným prístupom“)
REST	Representational State Transfer (slovensky „reprezentačný prenos stavu“)
RLHF	Reinforcement Learning from Human Feedback (slovensky „správaním posilňované učenie z ľudskej spätnej väzby“)
SaaS	Software as a Service (slovensky „softvér ako služba“)
SCADA	Supervisory Control and Data Acquisition (slovensky „dohľadové riadenie a zber dát“)
TCP	Transmission Control Protocol (slovensky „protokol riadenia prenosu“)
UI	User Interface (slovensky „používateľské rozhranie“)
VRAM	Video Random Access Memory (slovensky „video pamäť s náhodným prístupom“)

Zoznam algoritmov

1	Vypočítaj $y = x^n$	45
---	-------------------------------	----

Zoznam výpisov kódov

1	Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra <code>\lstinputlisting</code>	46
---	--	----

Úvod

Písanie záverečnej práce je nevyhnutnou súčasťou štúdia na vysokej škole. Študenti sa s touto činnosťou často stretávajú po prvýkrát na konci bakalárskeho štúdia. Príprava obsiahlejšieho textu je náročná úloha, ktorá zväčša trvá niekoľko týždňov alebo mesiacov. Pri finalizácii a formátovaní do výslednej podoby často panuje neistota a medzi študentami sa šíria protichodné informácie. Tento dokument slúži ako odporúčaná šablóna pre všetkých študentov FEI STU. Môžu ju využiť v plnej miere, alebo len ako inšpiráciu vytvorenia vlastnej výslednej podoby. Zároveň obsahuje návody a rady, ako sa vysporiadať s problémami, ktoré sa môžu pri písaní práce vyskytnúť.

Dokument je vytvorený v systéme na sadzbu textu $\text{\LaTeX}$ použitím upravenej verzie obľúbenej šablóny `FEIstyle`, verzia 2.0.0. Príručka je po formálnej a vizuálnej stránke navrhnutá tak, aby zodpovedala požiadavkám kladeným na záverečnú prácu vrátane poradia jednotlivých častí. Niektoré časti sú povinné (napr. obálka, titulný list, abstrakt), iné sú voliteľné (zoznam obrázkov, tabuliek, poďakovanie). Odporúčame študentom prečítať si aspoň prvú kapitolu tejto príručky, aby získali istotu pri začatí práce.

Jedným zo zámerov tohto manuálu je, aby jeho zdrojové súbory slúžili ako šablóna, do ktorej študenti vložia obsah vlastnej práce. Okrem definície rozmerov, okrajov, číslovania strán, štýlov a znakov, šablóna `FEIstyle.cls` obsahuje množstvo makier a príkazov na generovanie zoznamov obrázkov, tabuliek, výpisov kódov, algoritmov alebo obsahu. Zoznam a opis všetkých funkcií šablóny nájdete v priloženom dokumente `tutorial.pdf`.

Je žiadúce, aby študenti ovládali prácu s $\text{\LaTeX}$ -om na pokročilej užívateľskej úrovni. Je vhodné, ak si pred začiatkom písania preštudujú základné príručky. Veľmi vhodná je kniha Jiřího Rybyčku $\text{\LaTeX}$ pro začátečníky z roku 2003 [1]. Ide síce o staršiu publikáciu v českom jazyku, patrí však k vyhľadávaným didaktickým textom a je oceňovaná všetkými používateľmi $\text{\LaTeX}$ -u najmä pri zvládaní prvých krokov. Prehľadnou a príťažlivou formou je tiež spracovaný tutoriál pre používateľov internetovej služby Overleaf.¹ Je to bezplatný výkonný online kompilátor $\text{\LaTeX}$ -u podporovaný mnohými vedeckými vydavateľstvami.

V texte tejto rozšírenej šablóny nájdeme okrem iného aj poznámky k jazykovej stránke textu záverečných prác, písaniu matematických vzorcov, vytváraniu a formátovaniu grafov, tabuliek, či výpisu kódov programov.

V závere ?? kapitoly rozoberieme aktuálnu problematiku používania umelej inteligencie pri tvorbe akademických prác.

¹www.overleaf.com/learn

Citáciám a bibliografickým zdrojom venujeme celú kapitolu ??, pretože práca s externými zdrojmi je pri akademických textoch veľmi dôležitá. Záverečná práca má byť samostatné dielo študenta, ktorý čerpá poznatky z iných zdrojov, spracováva ich a spája do nových súvislostí, čím vytvára novú kvalitu. Veríme, že tento dokument študentom pomôže lepšie sa orientovať, dodá im istotu pri písaní, zabezpečí kvalitný výsledok, estetický vzhľad a hodnotný obsah. Táto príručka nie je komplexným nástrojom na všetky aspekty záverečnej práce, no poskytuje pevný základ úspešného písania a efektívnej práce s literatúrou.

Naším cieľom bolo vytvoriť materiál, ktorý vedie študenta pri hľadaní ďalších zdrojov v literatúre a na internete, aby sa vedel orientovať v terminológii a získal základný prehľad. Prajeme úspešné písanie a dosiahnutie všetkých stanovených cieľov.

1 Analýza existujúcich riešení

Trh s priemyselnými softvérovými platformami ponúka viacero riešení umožňujúcich vizualizáciu a ovládanie výrobných zariadení. Väčšina z nich vychádza z tradičného modelu Supervisory Control and Data Acquisition (slovensky „dohľadové riadenie a zber dát“) (SCADA)/Human-Machine Interface (slovensky „rozhranie človek-stroj“) (HMI) a postupne sa rozširuje o mobilný prístup, cloudovú konektivitu a v niektorých prípadoch aj o prvky umelej inteligencie. Nasledujúci prehľad analyzuje štyri významné komerčné platformy z pohľadu funkcionalít relevantných pre mobilné ovládanie priemyselných liniek: mobilný prístup k zariadeniam, identifikácia fyzických zariadení, nasadenie na edge vrstvu, otvorenosť platformy a integrácia inteligentného asistenta schopného pracovať s technickou dokumentáciou.

1.1 Siemens WinCC Unified

WinCC Unified je vizualizačná platforma od spoločnosti Siemens integrovaná do prostredia TIA Portal. Predstavuje natívne riešenie pre zariadenia radu SIMATIC a poskytuje webový HMI prístup prostredníctvom HyperText Markup Language (slovensky „hypertextový značkovací jazyk“) (HTML)⁵, čo umožňuje zobrazenie operátorských obrazoviek na mobilných zariadeniach cez webový prehliadač.

Platforma disponuje integráciou s Programmable Logic Controller (slovensky „programovateľný logický automat“) (PLC) radu S7-1200 a S7-1500 vrátane priameho prístupu k dátovým blokom a alarmom. Runtime beží priamo na operátorských paneloch Siemens alebo na priemyselných PC. Webový server umožňuje vzdialený prístup z mobilných zariadení, avšak rozhranie je primárne optimalizované pre pevné panely, nie pre mobilnú interakciu.

- **Mobilný prístup:** Webové rozhranie prístupné cez prehliadač; nie je natívna mobilná aplikácia.
- **Identifikácia zariadení:** Nie je podporovaná – operátor naviguje cez menu alebo prepína medzi obrazovkami manuálne.
- **Edge nasadenie:** Podporované cez SIMATIC Edge platformu, avšak vyžaduje proprietárny hardvér a licencie.
- **AI asistent:** Nie je súčasťou platformy. Siemens ponúka AI funkcie v rámci platformy Industrial Copilot, avšak ten je zameraný na generovanie PLC kódu v TIA Portal, nie na asistenta pre obsluhu s prístupom k technickej dokumentácii zariadení.

- **Licenčný model:** Proprietárny, viazaný na ekosystém Siemens.

1.2 Ignition Perspective

Ignition je modulárna SCADA platforma od spoločnosti Inductive Automation, ktorá si v posledných rokoch vybudovala silné postavenie vďaka flexibilnému licenčnému modelu a modernej architektúre. Modul Perspective poskytuje plnohodnotné mobilné rozhranie realizované ako webová aplikácia s responzívnym dizajnom optimalizovaným pre tablety a smartfóny.

Platforma komunikuje s PLC prostredníctvom Open Platform Communications Unified Architecture (slovensky „zjednotená architektúra otvorenej platformovej komunikácie“) (OPC UA), Modbus Transmission Control Protocol (slovensky „protokol riadenia prenosu“) (TCP) a ďalších priemyselných protokolov. Ignition Gateway beží na štandardnom serveri a je škálovateľný od jedného zariadenia po celé podniky prostredníctvom architektúry hub-and-spoke.

- **Mobilný prístup:** Modul Perspective poskytuje natívne mobilné rozhranie; k dispozícii je aj dedikovaná mobilná aplikácia Ignition Perspective App.
- **Identifikácia zariadení:** Nie je natívne podporovaná. Je možné implementovať Quick Response (slovensky „rýchla odozva“) (QR) kód navigáciu pomocou vlastných skriptov, avšak nejde o štandardnú funkciu platformy.
- **Edge nasadenie:** Ignition Edge je odľahčená verzia určená pre nasadenie na edge zariadeniach s obmedzenými zdrojmi.
- **AI asistent:** Nie je súčasťou platformy. Ignition disponuje výkonným skriptovacím rozhraním v Pythone, cez ktoré by bolo technicky možné integrovať volania na externé Large Language Model (slovensky „veľký jazykový model“) (LLM) Application Programming Interface (slovensky „aplikačné programové rozhranie“) (API), avšak platforma neponúka žiadnu vstavanú funkciu inteligentného asistenta ani správu znalostnej bázy.
- **Licenčný model:** Licencia na Gateway bez obmedzenia počtu klientov, čo je výrazne priaznivejšie oproti licenciám na stanicu. Moduly Perspective, Alarming, Reporting sa licencujú samostatne.

1.3 Tulip

Tulip je cloudová no-code platforma pre výrobný sektor, ktorá umožňuje vytvárať priemyselné aplikácie prostredníctvom vizuálneho drag-and-drop editora bez nutnosti

programovania. Platforma je orientovaná na digitalizáciu manuálnych výrobných procesov – sledovanie výrobných krokov, zber dát z pracovísk, kontrolu kvality a vizuálne pracovné inštrukcie.

Tulip komunikuje s priemyselnými zariadeniami prostredníctvom vlastného edge zariadenia (Tulip Edge MC alebo Tulip Player), ktoré podporuje OPC UA, Modbus a HyperText Transfer Protocol (slovensky „hypertextový prenosový protokol“) (HTTP) integrácie. Aplikácie bežia vo webovom prehliadači na tabletoch umiestnených priamo na pracoviskách.

- **Mobilný prístup:** Aplikácie sú webové a responzívne, primárne navrhnuté pre tablety na pracoviskách.
- **Identifikácia zariadení:** Podporuje skenovanie čiarových kódov a QR kódov pre identifikáciu materiálov a produktov, avšak nie pre identifikáciu a spárovanie so zariadeniami výrobnéj linky.
- **Edge nasadenie:** Tulip Edge MC zabezpečuje lokálnu komunikáciu so zariadeniami, avšak aplikačná logika a dáta sú hostované v cloude.
- **AI asistent:** Tulip v roku 2024 predstavil funkciu Frontline Copilot, ktorá využíva generatívnu AI na asistovanie operátorom. Podľa dostupných informácií sa jedná o integráciu s cloudovými LLM službami zameranú na analýzu výrobných dát a generovanie odporúčaní. Nejde však o asistenta s vlastnou znalostnou bázou technickej dokumentácie – funkcia pracuje primárne s dátami zozbieranými v rámci platformy Tulip.
- **Licenčný model:** Software as a Service (slovensky „softvér ako služba“) (SaaS) model s mesačnými platbami na používateľa. Cloudová závislosť môže byť problematická v prostrediach s prísnyimi požiadavkami na lokalizáciu dát.

1.4 AVEVA InTouch Edge HMI

AVEVA InTouch je jedna z najdlhšie pôsobiacich HMI/SCADA platforiem na trhu. Verzia Edge HMI je odľahčeným variantom určeným pre nasadenie na priemyselných PC a edge zariadeniach. Platforma poskytuje vizualizáciu procesných dát, správu alarmov a historické trendy.

InTouch Edge HMI komunikuje s PLC prostredníctvom vlastného komunikačného servera a podporuje širokú škálu priemyselných protokolov. Vzdialený webový prístup je možný prostredníctvom doplnkového modulu AVEVA Insight alebo webového servera.

- **Mobilný prístup:** Webový prístup cez cloudový portál AVEVA Insight alebo lokálny webový server. Nie je to natívna mobilná aplikácia pre priame ovládanie.
- **Identifikácia zariadení:** Nie je podporovaná.
- **Edge nasadenie:** Natívna podpora, InTouch Edge HMI je priamo navrhnutý pre beh na edge zariadeniach s minimálnymi hardvérovými nárokmi.
- **AI asistent:** AVEVA ponúka platformu AVEVA PI s analytickými funkciami a v roku 2024 predstavila AVEVA Industrial AI Assistant. Ten je však zameraný na analýzu procesných dát a prediktívnu údržbu na podnikovej úrovni, nie na kontextovú asistenciu obsluhu na základe technickej dokumentácie konkrétneho zariadenia.
- **Licenčný model:** Proprietárny, licencie na inštanciu a podľa počtu tagov.

Zhrnutie analýzy

Tabuľka 2 sumarizuje kľúčové vlastnosti porovnávaných platforiem vzhľadom na mobilný prístup, identifikáciu zariadení, edge nasadenie a integráciu AI asistenta s vlastnou znalostnou bázou.

Tabuľka 2: Porovnanie existujúcich riešení

Vlastnosť	WinCC Unified	Ignition Perspective	Tulip	AVEVA InTouch
Natívna mobilná aplikácia	Nie	Áno	Nie*	Nie
Identifikácia zariadení	Nie	Nie	Čiastočne**	Nie
Edge nasadenie	Áno	Áno	Čiastočne	Áno
AI asistent s vlastnou KB	Nie	Nie	Nie	Nie
Otvorená platforma	Nie	Čiastočne	Nie	Nie

* Webová aplikácia optimalizovaná pre tablety. ** QR/čiarové kódy pre materiály, nie pre zariadenia linky.

Z porovnania vyplýva, že existujúce komerčné platformy poskytujú rôznu mieru mobilného prístupu k priemyselným systémom, avšak ani jedna z nich neintegruje inteligentného asistenta schopného odpovedať na otázky obsluhy na základe vlastnej znalostnej bázy technickej dokumentácie. Väčšina platforiem sa orientuje na vizualizáciu a vzdialený prístup k procesným dátam. Pričom podpora operátora, teda schopnosť

systému aktívne pomáhať pri riešení prevádzkových situácií prostredníctvom práce s dokumentáciou, zostáva nepokrytou oblasťou.

Rovnako identifikácia fyzických zariadení prostredníctvom vizuálnych markerov s automatickým načítaním kontextu pre dané zariadenie nie je štandardnou funkciou žiadnej z analyzovaných platforiem. Operátori sú naďalej odkázaní na manuálnu navigáciu cez menu a obrazovky, čo spomaľuje prístup k informáciám a zvyšuje pravdepodobnosť interakcie s nesprávnym zariadením.

Tieto identifikované medzery v súčasnej ponuke komerčných riešení tvoria základ pre návrh systému prezentovaného v ďalších kapitolách tejto práce.

2 Prehľad technológií pre vývoj aplikácií a priemyselnú komunikáciu

Moderné priemyselné aplikácie nestoja na jednej technológii. Sú výsledkom starostlivo zvolenej skladačky nástrojov, kde každá vrstva plní svoju úlohu: mobilné rozhranie reaguje na pokyny operátora a backendová vrstva spracúva množstvo súbežných udalostí z výrobných linky.

Kľúčovou výzvou je pritom premostenie dvoch oddelených svetov. Operational Technology (slovensky „prevádzkové technológie“) (OT) označuje svet priemyselného riadenia a teda PLC automaty, senzory, pohony a výrobné linky, ktoré fyzicky ovládajú stroje v reálnom čase. Information Technology (slovensky „informačné technológie“) (IT) je naopak svet softvéru, sietí a aplikácií určených na spracovanie a prenos informácií. Tieto dva svety dlho fungovali oddelene, každý s vlastnými protokolmi a prioritami. Priemyselné aplikácie v kontexte Industry 4.0 ich spájajú práve prostredníctvom komunikačnej vrstvy, napríklad pomocou protokolov Message Queuing Telemetry Transport (slovensky „protokol na prenos telemetrických správ“) (MQTT) alebo OPC UA.

Nasledujúci prehľad mapuje najpoužívanjšie technológie v každej z týchto vrstiev a ich typické uplatnenie v priemyselnom prostredí.

2.1 Frontendové technológie

Pri výbere frontendovej technológie pre priemyselné aplikácie je kľúčové zvážiť cieľovú platformu (operátorský panel, tablet v teréne, manažérsky dashboard) a požiadavky na odozvu rozhrania pri spracovaní real-time dát.

2.1.1 Natívny vývoj

Natívny vývoj predstavuje prístup, kde je zdrojový kód písaný v jazyku a nástrojoch priamo podporovaných výrobcom operačného systému. Tento prístup využíva kompiláciu do strojového kódu danej architektúry, čo umožňuje priamu interakciu s jadrom systému bez sprostredkovateľských vrstiev.

- **Výhody:**

- **Absolútna optimalizácia:** Prístup k nízkoúrovňovým API umožňuje efektívnu správu pamäte a minimálne oneskorenie pri náročných operáciách.
- **Plná hardvérová integrácia:** Bezproblémové využitie senzorov, kamery pre rozšírenú realitu pri údržbe či technológií Bluetooth Low Energy pre lokalizáciu v halách.

- **Štandardizované UI:** Využitie systémových widgetov zaručuje konzistentnú používateľskú skúsenosť (UX).

- **Nevýhody:**

- **Fragmentácia vývoja:** Nutnosť paralelných tímov a duplicitná implementácia biznis logiky pre každú platformu zvyšuje TCO (Total Cost of Ownership).
- **Náročná údržba:** Synchronizácia verzií a opráv chýb naprieč ekosystémami je časovo nákladná.

2.1.2 React Native

React Native je multiplatformový framework od spoločnosti Meta, ktorý využíva paradigmu deklaratívneho programovania v jazyku JavaScript/TypeScript. Na rozdiel od hybridných webových aplikácií nevykresľuje rozhranie v prehliadači, ale cez špecializovanú vrstvu (Bridge alebo novšie JSI - JavaScript Interface) volá priamo natívne UI komponenty systému.

- **Výhody:**

- **Zdieľanie kódu:** Možnosť opätovného využitia až 90 % kódovej základne medzi Androidom a iOS, čo dramaticky zrýchľuje proces Time-to-Market.
- **Ekosystém NPM:** Prístup k tisícom knižníc pre správu stavu a vizualizáciu dát.
- **Nová architektúra (JSI):** Odstránenie asynchrónneho bridge-u v prospech priameho volania C++ objektov výrazne zlepšilo odozvu.

- **Nevýhody:**

- **Abstrakčná réžia:** Komplexné animácie alebo masívny tok dát z PLC môžu naraziť na limity výkonu JavaScript vlákna.
- **Zložitejšie ladenie:** Chyby sa môžu vyskytovať na rozhraní medzi JS a natívnou vrstvou, čo sťažuje diagnostiku.

2.1.3 Flutter

Flutter je moderný UI toolkit od spoločnosti Google určený na vývoj natívne kompilovaných aplikácií z jednotnej kódovej základne v jazyku Dart. Na rozdiel od iných frameworkov nevyužíva systémové komponenty, ale disponuje vlastným grafickým jadrom (Skia/Impeller), ktoré vykresľuje rozhranie pixel po pixeli priamo na GPU.

- **Výhody:**

- **Vizuálna identita:** Úplná kontrola nad každým pixelom zaručuje, že rozhranie bude vyzeráť identicky na starom operátorskom paneli aj novom tablete.
- **AOT kompilácia:** Zdrojový kód je dopredu kompilovaný do strojového kódu, čo eliminuje potrebu runtime interpretácie a zvyšuje výkon.
- **Hot Reload:** Schopnosť injektovať zmeny kódu do bežiacej aplikácie bez straty stavu urýchľuje prototypovanie priemyselných HMI.

- **Nevýhody:**

- **Veľkosť binárneho súboru:** Zahrnutie celého vykresľovacieho enginu do balíka aplikácie zvyšuje nároky na úložisko.
- **Špecifickosť jazyka:** Dart nie je v priemyselnom IT tak etablovaný ako C # alebo JavaScript, čo môže ovplyvniť dostupnosť vývojárov.

2.1.4 Webové aplikácie

Moderné webové frameworky umožňujú vytvárať komplexné klientske aplikácie, kde sa celá logika zobrazenia vykonáva v prehliadači používateľa. Využívajú techniku *Virtual DOM* (React/Vue) alebo *Incremental DOM* (Angular) na optimalizáciu prekresľovania len tých častí obrazovky, ktoré sa skutočne zmenili.

- **Výhody:**

- **Univerzálna dostupnosť:** Prístup k monitoringu cez akékoľvek zariadenie s moderným prehliadačom bez nutnosti inštalácie.
- **PWA (Progressive Web Apps):** Možnosť lokálneho cachovania a obmedzeného offline režimu, čo je užitočné pri výpadkoch siete v priemyselných prevádzkach.

- **Nevýhody:**

- **Bezpečnostný sandbox:** Prehliadač striktne obmedzuje prístup k súborovému systému a portom zariadenia, čo komplikuje priamu integráciu s lokálnym hardvérom.

2.2 Backendové technológie

Backend v priemyselnom kontexte funguje ako inteligentný middleware, ktorý sprostredkováva obojsmerný tok dát medzi deterministickým svetom PLC a dynamickým svetom používateľských aplikácií.

2.2.1 Node.js a NestJS

Node.js je behové prostredie postavené na engine V8, ktoré využíva neblokujúci, udalosťami riadený I/O model. NestJS nad ním buduje architektúru inšpirovanú Angularom (Dependency Injection, modulárnosť), čo prináša do JavaScriptového sveta poriadok potrebný pre enterprise systémy.

- **Výhody:**

- **I/O efektívnosť:** Ideálne pre aplikácie s vysokou frekvenciou krátkych správ, ako sú telemetry dáta z MQTT.
- **Isomorfizmus:** Možnosť zdieľať validačné schémy (napodobeniny dát z PLC) medzi backendom a frontendom cez TypeScript.

- **Nevýhody:**

- **Single-threaded limitácia:** CPU-intenzívne operácie (napr. komplexná analýza trendov) môžu zablokovat Event Loop a zvýšiť latenciu ostatných požiadaviek.

2.2.2 Python a FastAPI

Python je vďaka svojej syntaxi a obrovskej vedeckej základni dominantným jazykom pre prácu s dátami. FastAPI využíva moderné štandardy ako typové anotácie a ASGI na dosiahnutie vysokého výkonu pri zachovaní jednoduchosti.

- **Výhody:**

- **AI/ML integrácia:** Priama cesta od zberu dát z PLC k prediktívnej údržbe pomocou knižníc Scikit-learn alebo PyTorch.
- **Auto-dokumentácia:** Automatické generovanie interaktívnej dokumentácie (Swagger/OpenAPI), čo uľahčuje integráciu pre iných vývojárov.

- **Nevýhody:**

- **Interpretovaný charakter:** Aj napriek optimalizáciám je Python pomalší pri čistých matematických výpočtoch oproti kompilovaným jazykom.

2.2.3 Go

Go je kompilovaný, staticky typovaný jazyk navrhnutý v Google s dôrazom na jednoduchosť, bezpečnosť a masívny paralelizmus. Jeho kľúčovou vlastnosťou sú *goroutines* – extrémne ľahké vlákna spravované runtime-om jazyka.

- **Výhody:**

- **Statické binárky:** Výsledkom je jeden súbor obsahujúci všetky závislosti, čo je ideálne pre nasadenie na Edge brány bez potreby inštalovať runtime.
- **Minimálna réžia:** Nízka spotreba Random Access Memory (slovensky „pamäť s náhodným prístupom“) (RAM) a Central Processing Unit (slovensky „centrálne procesorová jednotka“) (CPU) robí z Go favorita pre mikroslužby bežiacie na limitovanom priemyselnom hardvéri.

- **Nevýhody:**

- **Menej abstrakcie:** Chýbajúca podpora pre niektoré pokročilé programovacie vzory (hoci generiká už boli pridané) môže viesť k redundantnejšiemu kódu.

2.3 Kontajnerizácia a Edge architektúra

V súčasnosti sa často spracovanie dát presúva z cloudu na **Edge** – k zdroju dát. Tento prístup znižuje latenciu, zvyšuje bezpečnosť (dáta neopúšťajú lokálnu sieť) a šetrí prenosové pásmo.

2.3.1 Docker a Docker Compose

Docker využíva technológie jadra Linuxu (namespaces, cgroups) na izoláciu procesov do tzv. kontajnerov. Na rozdiel od virtuálnych strojov kontajnery nezahrňujú celý operačný systém, ale zdieľajú jadro hostiteľa, čo ich robí extrémne ľahkými.

- **Kľúčový prínos pre priemysel:**

- **Prenositelnosť:** Docker balí aplikáciu aj s presnou verziou knižníc a systémových závislostí. To eliminuje problémy pri nasadzovaní na rôzne typy priemyselných PC (IPC).
- **Layered File System:** Vrstvený súborový systém umožňuje rýchle aktualizácie – pri zmene kódu sa prenáša len malá vrstva, nie celý obraz, čo je kľúčové pri aktualizácii cez LTE/IoT linky.

- **Docker Compose:** Umožňuje definovať celý aplikačný stack (napr. PostgreSQL pre dáta, Mosquitto pre správy a FastAPI pre logiku) v jednom konfiguračnom súbore, čím sa orchestrácia Edge uzla stáva triviálnou.
- **Výhody:** Rýchle nasadenie, jednoduchá horizontálna škálovateľnosť a izolácia porúch.
- **Nevýhody:** Vyžaduje správu obrazov a pochopenie sieťových vrstiev (bridge vs. host networking) pre komunikáciu s PLC.

2.4 Priemyselné komunikačné protokoly

2.4.1 OPC UA

OPC UA je platformovo nezávislý štandard, ktorý nahrádza staršie riešenia založené na Microsoft COM/DCOM. Definuje nielen transport, ale aj informačný model (uzly, objekty, premenné).

- **Výhody:** Vstavaná kybernetická bezpečnosť (šifrovanie, certifikáty), sémantický popis dát a podpora pre zložité hierarchické štruktúry.
- **Nevýhody:** Vysoká komplexnosť stacku a nároky na RAM/CPU v PLC.

2.4.2 S7 Protokol

Ide o natívnu komunikáciu pre PLC Siemens. Využíva ISO on TCP (RFC 1006) a umožňuje nízkoúrovňový prístup k pamäťovým oblastiam automatu.

- **Výhody:** Maximálna rýchlosť v ekosystéme Siemens bez nutnosti licencií pre OPC UA servery.
- **Nevýhody:** Chýbajúci sémantický kontext (vývojár musí presne poznať adresy DB blokov) a slabšie natívne šifrovanie.

2.4.3 MQTT

MQTT je ľahký binárny protokol typu publish-subscribe, optimalizovaný pre Internet of Things (slovensky „internet vecí“) (IoT). Správy sa neposielajú priamo, ale cez sprostredkovateľa (broker).

- **Výhody:** Minimálna veľkosť hlavičiek (2 bajty), podpora stavu „Last Will and Testament“ a rôznych úrovní doručenia (Quality of Service (slovensky „kvalita služieb“) (QoS)).
- **Nevýhody:** Nutnosť udržiavať bežiaci broker ako centrálny bod zlyhania (Single Point of Failure).

2.4.4 Modbus TCP

Najrozšírenejší "legacy" protokol založený na architektúre Client/Server. Pracuje s jednoduchým modelom registrov (Holding, Input) a cievok (Coils).

- **Výhody:** Extrémna jednoduchosť implementácie na takmer každom mikrokontroléri alebo PLC.
- **Nevýhody:** Žiadna bezpečnosť (všetko v plain-text) a obmedzené dátové typy (nutnosť manuálneho spájania 16-bitových registrov do float/int64).

3 Prehľad technológií pre inteligentných asistentov

Jedným z cieľov tejto práce je integrácia inteligentného technického asistenta do mobilnej aplikácie pre obsluhu výrobných linky. Asistent má byť schopný odpovedať na otázky operátorov na základe technickej dokumentácie konkrétnych zariadení, čím znižuje závislosť od papierových manuálov a skracuje čas potrebný na riešenie prevádzkových situácií. Realizácia tohto cieľa stojí na troch vzájomne prepojených pilieroch: veľkých jazykových modeloch, ktoré tvoria jadro generovania odpovedí; koncepte virtuálnych Artificial Intelligence (slovensky „umelá inteligencia“) (AI) asistentov, ktoré definujú spôsob interakcie s používateľom; a technike Retrieval-Augmented Generation (slovensky „generovanie rozšírené o vyhľadávanie“) (RAG), ktorá umožňuje modelu pracovať s externými znalosťami bez nutnosti pretrénovania.

3.1 Veľké jazykové modely - LLM

3.1.1 Definícia a princíp fungovania

Veľké jazykové modely, angl. Large Language Models, sú neurónové siete založené na architektúre Transformer, ktoré sú trénované na rozsiahlych textových korpusoch s cieľom modelovať štatistické rozloženie jazyka. Základným princípom je predikcia nasledujúceho tokenu. Model na základe doterajšieho kontextu odhaduje pravdepodobnosť každého možného pokračovania a generuje text token po tokene.

Proces tréningu LLM prebieha typicky v dvoch fázach:

1. **Predtrénovanie (pre-training):** Model sa trénuje na obrovských objemoch neoznačeného textu (stovky miliárd až miliardy tokenov) úlohou predikcie nasledujúceho tokenu (causal language modeling). V tejto fáze model získava jazykové znalosti, faktické informácie a schopnosť uvažovania. Predtrénovanie vyžaduje enormné výpočtové zdroje – tréning najväčších modelov spotrebuje tisíce Graphics Processing Unit (slovensky „grafická procesorová jednotka“) (GPU)-hodín.
2. **Doladenie (fine-tuning):** Predtrénovaný model sa ďalej prispôbuje na špecifickú úlohu alebo správanie. Najčastejšie sa využíva *instruction tuning*, kde sa model trénuje na pároch inštrukcia–odpoveď, a Reinforcement Learning from Human Feedback (slovensky „správaním posilňované učenie z ľudskej spätnej väzby“) (RLHF), kde ľudskí hodnotitelia posudzujú kvalitu odpovedí a model sa učí generovať odpovede preferované ľuďmi.

3.1.2 Kľúčové parametre a vlastnosti

Výkon a správanie LLM determinuje niekoľko kľúčových parametrov:

- **Počet parametrov:** Označuje celkový počet trénovateľných váh v neurónovej sieti. Modely sa pohybujú od jednotiek miliárd (napr. Llama 3.2 s 3B) cez desiatky miliárd (Gemma 2 s 9B, Mistral Small s 24B) až po stovky miliárd parametrov (GPT-4, Claude). Vo všeobecnosti platí, že väčší model má vyššiu kapacitu pre zachytenie jazykových a faktických vzorcov, avšak tento vzťah nie je lineárny – kvalita trénovacích dát a architektúra môžu tento efekt výrazne modulovať.
- **Kontextové okno (context window):** Maximálny počet tokenov, ktoré model dokáže spracovať v jednej výzve (prompt). Tento parameter je kritický pre aplikácie typu RAG, pretože priamo limituje objem kontextu, ktorý je možné modelu poskytnúť spolu s otázkou.
- **Tokenizácia:** Spôsob, akým model rozdeľuje vstupný text na tokeny, priamo ovplyvňuje jeho efektívnosť pri rôznych jazykoch. Modely využívajúce tokenizéry s väčším slovníkom typicky efektívnejšie kódujú ne-anglické jazyky vrátane slovenčiny, pretože potrebujú menej tokenov na reprezentáciu rovnakého textu.
- **Teplota (temperature):** Parameter ovplyvňujúci mieru náhodnosti pri generovaní. Nízka teplota (blízka 0) produkuje deterministické, fakticky konzistentné odpovede, zatiaľ čo vyššia teplota (0,7–1,0) zvyšuje kreativitu a variabilitu. Pre technického asistenta v priemyselnom prostredí je žiaduca nízka teplota, pretože prioritou je faktická presnosť.

3.1.3 Otvorené vs. proprietárne modely

Z hľadiska dostupnosti a nasadenia rozlišujeme dve kategórie LLM:

Proprietárne modely (GPT-4 od OpenAI, Claude od Anthropic, Gemini od Google) sú prístupné výhradne prostredníctvom cloudových API. Ponúkajú spravidla najvyšší výkon, avšak ich používanie vyžaduje odosielanie dát na externé servery, čo môže byť v priemyselnom prostredí neprijateľné z hľadiska bezpečnosti a ochrany dát. Prevádzka je viazaná na internetové pripojenie a spoplatňovaná podľa objemu spracovaných tokenov.

Otvorené modely (Llama od Meta, Gemma od Google, Mistral, Phi od Microsoft) je možné stiahnuť a prevádzkovať lokálne na vlastnom hardvéri. Tento prístup eliminuje závislosť od externých služieb, zabezpečuje, že citlivé prevádzkové dáta neopúšťajú lokálnu sieť, a umožňuje prevádzku bez internetového pripojenia. Nevýhodou sú vyššie

nároky na hardvér (najmä GPU s dostatočnou Video Random Access Memory (slovensky „video pamäť s náhodným prístupom“) (VRAM)) a spravidla nižší výkon v porovnaní s najväčšími proprietárnymi modelmi.

3.1.4 Platforma Ollama

Ollama je open-source platforma pre lokálne spúšťanie a správu veľkých jazykových modelov. Poskytuje jednotné rozhranie pre stiahnutie, spustenie a komunikáciu s modelmi rôznych architektúr prostredníctvom Representational State Transfer (slovensky „reprezentačný prenos stavu“) (REST) API kompatibilného s formátom OpenAI. Ollama automaticky spravuje kvantizáciu modelov, alokáciu pamäte medzi GPU a systémovou RAM a poskytuje jednoduché Command Line Interface (slovensky „rozhranie príkazového riadku“) (CLI) rozhranie pre správu modelov.

Z pohľadu integrácie do aplikačného stacku je kľúčovou vlastnosťou Ollama práve REST API – backend v Pythone (FastAPI) komunikuje s modelom cez HTTP požiadavky na lokálny endpoint, čo umožňuje jednoduché prepínanie medzi modelmi bez zmeny aplikačného kódu. Knižnice ako LangChain poskytujú natívnu integráciu s Ollama, čo ďalej zjednodušuje implementáciu RAG pipeline.

3.2 Virtuálni AI asistenti

3.2.1 Definícia a klasifikácia

Virtuálny AI asistent je softvérový systém schopný viesť konverzáciu v prirodzenom jazyku s cieľom poskytnúť používateľovi informácie, vykonať úlohy alebo asistovať pri rozhodovaní. Na rozdiel od tradičných chatbotov založených na pravidlách (rule-based), moderní AI asistenti využívajú veľké jazykové modely, čo im umožňuje rozumieť kontextu, spracovať neštruktúrované otázky a generovať koherentné odpovede aj na otázky, ktoré neboli explicitne naprogramované.

Podľa architektúry a schopností je možné AI asistentov klasifikovať do niekoľkých úrovní:

1. **Retrieval-based asistenti:** Odpoveď vyberajú z predpripravenej databázy odpovedí na základe podobnosti s otázkou. Sú spoľahliví, ale obmedzení na vopred definované scenáre.
2. **Generatívni asistenti:** Využívajú LLM na generovanie odpovedí. Sú flexibilní, ale môžu produkovať fakticky nesprávne informácie (halucinácie).

3. **RAG-augmentovaní asistenti:** Kombinujú generatívny model s vyhľadávaním v externej znalostnej báze. Tento prístup výrazne znižuje halucinácie a umožňuje pracovať s aktuálnymi, doménovo špecifickými informáciami.
4. **Agentní asistenti:** Okrem generovania textu dokážu vykonávať akcie – volať API, spúšťať nástroje, pristupovať k databázam. Reprezentujú najvyspelejšiu kategóriu.

3.2.2 Architektúra konverzačného AI asistenta

Typická architektúra konverzačného AI asistenta pozostáva z niekoľkých vrstiev, ktoré spolupracujú pri spracovaní každej používateľskej otázky:

Vstupná vrstva prijíma otázku od používateľa a vykonáva jej predspracovanie – normalizáciu textu, detekciu jazyka a prípadnú reformuláciu otázky pre lepšie vyhľadávanie v znalostnej báze.

Retrieval vrstva na základe spracovanej otázky vyhľadá relevantné dokumenty alebo fragmenty textu z externej znalostnej bázy. Táto vrstva je kľúčová pre RAG architektúru a jej kvalita priamo determinuje kvalitu výslednej odpovede.

Generatívna vrstva prijíma pôvodnú otázku spolu s nájdenými dokumentmi ako kontext a prostredníctvom LLM generuje odpoveď. Systémový prompt definuje správanie asistenta – jazyk odpovede, rozsah tém, formát a inštrukcie pre prípad, keď znalostná báza neobsahuje relevantnú odpoveď.

Výstupná vrstva formátuje odpoveď pre zobrazenie v používateľskom rozhraní a prípadne vykonáva post-processing ako filtrovanie nevhodného obsahu alebo pridanie referencií na zdrojové dokumenty.

3.2.3 Systémový prompt a kontrola správania

Systémový prompt (system prompt) je inštrukcia predradená pred používateľskú otázku, ktorá definuje identitu, správanie a obmedzenia asistenta. V kontexte priemyselného asistenta systémový prompt plní niekoľko kritických funkcií:

- **Vymedzenie rozsahu:** Inštruuje model, aby odpovedal výhradne na základe poskytnutého kontextu a odmietol otázky mimo rozsahu znalostnej bázy. Tým sa predchádza generovaniu fakticky nesprávnych informácií, ktoré by v priemyselnom prostredí mohli viesť k nesprávnym rozhodnutiam obsluhy.
- **Jazyková konzistencia:** Definuje, v akom jazyku má asistent odpovedať, prípadne inštruuje model, aby odpovedal v rovnakom jazyku, v akom bola položená otázka.

- **Bezpečnostné ohraňenie:** Zabraňuje modelu poskytovať rady, ktoré by mohli viesť k poškodeniu zariadenia alebo ohrozeniu bezpečnosti obsluhy, a smeruje operátora na kontaktovanie technika v prípade neistoty.
- **Formát odpovede:** Špecifikuje požadovanú štruktúru odpovede – stručnosť, používanie odrážok, prípadne krokové inštrukcie pre procedurálne otázky.

3.2.4 Výzvy a obmedzenia v priemyselnom prostredí

Nasadenie AI asistenta v priemyselnom prostredí prináša špecifické výzvy, ktoré sa v bežných konverzačných scenároch nevyskytujú:

Faktická spoľahlivosť je v priemysle kritická. Nesprávna odpoveď o postupe údržby alebo nastavení parametrov stroja môže viesť k materiálnym škodám alebo bezpečnostným incidentom. Preto je nevyhnutné implementovať mechanizmy na minimalizáciu halucinácií – najmä techniku RAG a striktný systémový prompt.

Jazyková diverzita predstavuje výzvu v multijazyčnom priemyselnom prostredí. Technická dokumentácia môže byť v angličtine, zatiaľ čo operátori komunikujú v slovenčine. Model musí byť schopný pracovať s viacjazyčným kontextom a odpovedať v jazyku otázky.

Latencia je dôležitým faktorom pri interaktívnom použití. Operátor na výrobnéj linke očakáva odpoveď v rádoch sekúnd, nie minút. To kladie požiadavky na výber dostatočne rýchleho modelu a optimalizáciu retrieval pipeline.

Bezpečnosť dát je dôležitou požiadavkou, avšak jej rozsah závisí od konkrétnej implementácie. V prípade využitia RAG architektúry je kľúčové rozlišovať, aké dáta skutočne opúšťajú lokálnu sieť. Samotná znalostná báza a vektorová databáza zostávajú uložené lokálne – do cloudu sa odosiela iba zostavený prompt obsahujúci vyhľadane fragmenty dokumentácie a otázku operátora, nie surové prevádzkové dáta zo zariadení (parametre procesov, údaje o poruchách). Pri správnom návrhu systému je preto možné využiť cloudovú LLM službu bez toho, aby citlivé prevádzkové informácie opúšťali lokálnu infraštruktúru.

3.3 RAG

3.3.1 Motivácia a princíp

Veľké jazykové modely disponujú rozsiahlymi znalosťami získanými počas predtrénovania, avšak tieto znalosti majú niekoľko zásadných obmedzení: sú zmrazené k dátumu ukončenia tréningu (knowledge cutoff), nemusia pokrývať úzko špecializované domény a model nemá možnosť rozlíšiť, kedy si je odpoveďou istý a kedy halucinuje.

RAG je technika, ktorá tieto obmedzenia adresuje doplnením externého zdroja znalostí do inferenčného procesu. Namiesto spolieha sa výhradne na parametrické znalosti modelu, RAG systém pred generovaním odpovede vyhľadá relevantné dokumenty z externej znalostnej bázy a poskytne ich modelu ako súčasť vstupného kontextu. Model tak generuje odpoveď na základe kombinácie svojich jazykových schopností a aktuálnych, overiteľných informácií.

3.3.2 Architektúra RAG pipeline

Kompletný RAG systém pozostáva z niekoľkých komponentov, ktoré spolupracujú v dvoch fázach: offline príprava znalostnej bázy (indexácia) a online spracovanie otázky (inferencia).

Fáza indexácie

1. **Načítanie dokumentov (document loading):** Zdrojové dokumenty (technické manuály, prevádzkové predpisy, schémy zapojenia) sa načítajú z rôznych formátov – PDF, TXT, DOCX, HTML. V kontexte priemyselného asistenta je kritické zachovať štruktúru dokumentov (nadpisy, sekcie, tabuľky), pretože táto štruktúra nesie sémantickú informáciu.
2. **Delenie na fragmenty (chunking):** Dokumenty sa rozdelia na menšie časti (chunks) vhodné pre vektorové vyhľadávanie. Veľkosť fragmentu je kľúčovým parametrom – príliš malé fragmenty strácajú kontext, príliš veľké znižujú presnosť retrieval vrstvy a zaberajú väčšiu časť kontextového okna modelu. Bežne sa používajú fragmenty o veľkosti 256–1024 tokenov s prekryvom (overlap) 10–20 % pre zachovanie kontextu na hraniciach.
3. **Vektorizácia (embedding):** Každý fragment sa transformuje na viacrozmerný číselný vektor (embedding) pomocou embedding modelu. Embedding model je špecializovaná neurónová sieť trénovaná tak, aby sémanticky podobné texty mali blízke vektory v priestore. Výsledný vektor typicky obsahuje 384–1536 dimenzií.
4. **Uloženie do vektorovej databázy:** Vektory spolu s pôvodným textom fragmentov sa uložia do vektorovej databázy, ktorá umožňuje efektívne vyhľadávanie najbližších susedov (nearest neighbor search).

Fáza inferencie Pri prijatí otázky od používateľa prebieha spracovanie v nasledujúcich krokoch:

1. **Vektorizácia otázky:** Otázka sa transformuje na vektor rovnakým embedding modelom, aký bol použitý pri indexácii dokumentov. Konzistencia embedding

modelu medzi indexáciou a dopytovaním je nevyhnutná pre správne fungovanie vyhľadávania.

2. **Vyhľadanie relevantných fragmentov:** Vo vektorovej databáze sa vyhľadajú fragmenty, ktorých vektory sú najbližšie k vektoru otázky, podľa zvolenej metriky podobnosti (kosínusová podobnosť, euklidovská vzdialenosť). Počet vrátených fragmentov (parameter k) je kompromis medzi pokrytím informácií a efektívnym využitím kontextového okna modelu.
3. **Zostavenie promptu:** Systémový prompt, nájdené fragmenty (ako kontext) a pôvodná otázka sa zostavia do finálneho promptu pre jazykový model. Typický formát:

```
[Systémový prompt: Si technický asistent PLEXO...]  
[Kontext: {fragment_1} {fragment_2} ... {fragment_k}]  
[Otázka: {otázka_používateľa}]
```

4. **Generovanie odpovede:** Jazykový model spracuje zostavený prompt a vygeneruje odpoveď. Vďaka poskytnutému kontextu model nemusí spoliehať na svoje parametrické znalosti, ale odpovedá na základe konkrétnych informácií z dokumentácie.

3.3.3 Vektorové databázy a embedding modely

Vektorová databáza je špecializovaný databázový systém optimalizovaný pre ukladanie a vyhľadávanie viacrozmerných vektorov. Na rozdiel od relačných databáz, kde sa vyhľadáva podľa presných hodnôt alebo rozsahov, vektorové databázy vyhľadávajú podľa *podobnosti* – nájdu záznamy, ktorých vektory sú najbližšie k vektoru dopytu.

Najpoužívanejšie algoritmy pre aproximované vyhľadávanie najbližších susedov (Approximate Nearest Neighbor (slovensky „vyhľadávanie aproximovaných najbližších susedov“) (ANN)) zahŕňajú:

- **Hierarchical Navigable Small World (slovensky „hierarchicky navigovateľné malé svety“) (HNSW)** – grafový algoritmus s logaritmickou zložitostou vyhľadávania, používaný v ChromaDB, Qdrant a Weaviate.
- **Inverted File Index (slovensky „index invertovaných súborov“) (IVF)** – rozdeľuje vektorový priestor na klastry a vyhľadáva len v najbližších klastroch, používaný v FAISS od Meta.

- **Product Quantization** – komprimuje vektory pre úsporu pamäte pri zachovaní prijateľnej presnosti.

ChromaDB, použitý v tejto práci, je open-source vektorová databáza navrhnutá pre jednoduchú integráciu s LLM aplikáciami. Podporuje perzistentné uloženie na disk, filtrovanie výsledkov podľa metadát (napr. typ dokumentu, jazyk, dátum) a priamu integráciu s frameworkom LangChain. Pre priemyselného asistenta je výhodou jej nízka prevádzková náročnosť – beží ako knižnica v rámci Python procesu bez nutnosti prevádzkovať samostatný server. ChromaDB beží výhradne lokálne a žiadne dáta z nej neopúšťajú lokálnu sieť.

Embedding model transformuje text na hustý vektor s pevnou dimenziou. Kvalita embedding modelu priamo ovplyvňuje kvalitu retrieval vrstvy – ak model nedokáže zachytiť sémantickú podobnosť medzi otázkou a relevantným dokumentom, RAG systém zlyhá bez ohľadu na kvalitu jazykového modelu. Model `nomic-embed-text`, použitý v tejto práci, je otvorený embedding model prevádzkovaný lokálne cez platformu Ollama. Vektorizácia prebieha teda výhradne na lokálnom zariadení – do cloudu sa odosiela až výsledný zostavený prompt, nie samotné dokumenty znalostnej bázy.

3.3.4 Framework LangChain

LangChain je open-source framework pre budovanie aplikácií založených na jazykových modeloch. Poskytuje abstrakcie pre všetky komponenty RAG pipeline – načítanie dokumentov (document loaders), delenie textu (text splitters), vektorizáciu (embeddings), vektorové databázy (vector stores) a reťazenie operácií (chains).

Kľúčovou výhodou LangChain je modulárna architektúra, ktorá umožňuje zameniť ľubovoľný komponent bez zmeny zvyšku pipeline. Napríklad prechod z embedding modelu `nomic-embed-text` na iný model vyžaduje zmenu jedného riadku konfigurácie. Rovnako je možné zmeniť vektorovú databázu z ChromaDB na Qdrant alebo FAISS bez prepísania retrieval logiky.

Pre integráciu s Claude API poskytuje LangChain triedu `ChatAnthropic`, ktorá odosiela zostavený prompt na Anthropic API a vracia vygenerovanú odpoveď. Embedding vrstva zostáva lokálna – využíva triedu `OllamaEmbeddings` s modelom `nomic-embed-text` a vektorovú databázu **Chroma**. Vďaka modulárnej architektúre LangChain stačí zameniť jedinou triedu (z `ChatAnthropic` na `ChatOllama`) a bez zásahu do zvyšku RAG pipeline vieme zmeniť implementáciu na lokálny model od Ollama v prípade takejto požiadavky.

3.3.5 Výhody a obmedzenia RAG

Výhody techniky RAG v porovnaní s čistým generatívnym prístupom:

- **Redukcia halucinácií:** Model generuje odpovede na základe konkrétnych dokumentov, nie len na základe parametrických znalostí. Ak znalostná báza neobsahuje relevantnú informáciu, správne nakonfigurovaný systém túto skutočnosť prizná.
- **Aktualizovateľnosť:** Pridanie novej dokumentácie nevyžaduje pretrénovanie modelu – stačí pridať dokumenty do vektorovej databázy. V priemyselnom kontexte to znamená, že pri inštalácii nového zariadenia postačí nahráť jeho manuál.
- **Transparentnosť:** Je možné zobrazíť, z ktorých dokumentov asistent čerpal, čo zvyšuje dôveryhodnosť odpovede a umožňuje operátorovi overiť informáciu v pôvodnom zdroji.
- **Doménová špecializácia:** Bez RAG by model odpovedal na základe všeobecných znalostí z predtrénovania. S RAG odpovedá na základe konkrétnej technickej dokumentácie relevantnej pre dané zariadenie a prevádzku.

Obmedzenia RAG, ktoré je potrebné zohľadniť pri návrhu systému:

- **Kvalita retrieval vrstvy:** Ak vyhľadávanie nevráti relevantné dokumenty, model nemá k dispozícii správny kontext a kvalita odpovede klesá. Kvalita závisí od embedding modelu, stratégie delenia dokumentov a formulácie otázky.
- **Obmedzenosť kontextového okna:** Počet fragmentov, ktoré je možné poskytnúť modelu, je limitovaný veľkosťou kontextového okna. Pri veľkých znalostných bázach môže byť potrebné sofistikovanejšie stratégie (re-ranking, hierarchický retrieval).
- **Závislosť na kvalite dokumentácie:** RAG systém je len taký dobrý, aká je znalostná báza. Nekompletná, zastaraná alebo nekonzistentná dokumentácia sa prejaví v kvalite odpovedí asistenta.

4 Typy identifikátorov a ich použitie

4.1 QR kódy

QR kódy sú dvojrozmerné maticové identifikátory optimalizované pre rýchle čítanie a vysokú hustotu dát. V kontexte priemyselnej automatizácie sa využívajú najmä na statickú identifikáciu objektov, kde sa vyžaduje uloženie komplexnejších informácií napr. URL adresy, sériové čísla alebo konfiguračné parametre. Ich výhodou je silná korekcia chýb, ktorá umožňuje prečítať kód aj pri čiastočnom poškodení alebo znečistení povrchu.

4.2 Fiduciálne markery - AprilTag, ArUco, STag

Fiduciálne markery sú špeciálne navrhnuté pre potreby počítačového videnia a robotiky. Na rozdiel od QR kódov nie je ich primárnym cieľom prenos veľkého objemu dát, ale presná lokalizácia v 3D priestore.

Tieto identifikátory sa vyznačujú predovšetkým vysokou mierou interoperability s modernými robotickými riešeniami. Vďaka existujúcim knižniciam napr. OpenCV, ROS - Robot Operating System je ich integrácia štandardom pri vývoji:

- **Autonómnych upratovacích robotov:** Kde markery umiestnené v prostredí slúžia ako pevné referenčné body pre rekalibráciu lokalizácie a elimináciu kumulatívnej chyby v mapách.
- **Dronov v skladoch:** Kde umožňujú presné pristávanie na nabíjaciach staniciach alebo navigáciu v priestoroch bez signálu GPS, pričom dron dokáže v reálnom čase určiť svoju polohu a náklon voči markeru.
- **Kolaboratívnych robotov:** Pri navádzaní na presnú pozíciu.

Identifikátory ako AprilTag a ArUco sú robustné voči zmenám osvetlenia a perspektívnemu skresleniu, zatiaľ čo STag (Stable Tag) prináša zvýšenú stabilitu a presnosť pri určovaní orientácie.

4.3 3D identifikátory

V prostrediach priemyselných podnikov a logistických reťazcov, kde sú si entity tvarovo a rozmerovo blízke, sa ako perspektívne riešenie javia unikátne 3D identifikátory. Na Fakulte elektrotechniky a informatiky STU bol vyvinutý systém programaticky generovaných 3D identifikátorov, ktoré sú optimalizované pre systémy zmiešanej a rozšírenej reality.

4.3.1 Princíp a štruktúra

Navrhnutý 3D identifikátor má tvar kocky, ktorá je vnútorne vyskladaná z menších, unikátnych geometrických elementov. Tieto elementy sú usporiadané v originálnej štruktúre, ktorá v sebe nesie zakódované informácie.

Hlavné technické výhody tohto riešenia sú:

- **Všesmerová detekcia:** Na rozdiel od 2D QR kódov je možné tieto 3D objekty v systémoch zmiešanej reality spoľahlivo snímať z akéhokoľvek uhla.
- **Odolnosť voči falšovaniu:** Vďaka komplexnej vnútornej členitosti a 3D geometrii sú tieto identifikátory výrazne náročnejšie na replikáciu než bežné 2D identifikátory.
- **Využitie vlastného tvaru:** Sledovanie sa nespolieha len na vizuálny vzor, ale na obrys a priestorovú charakteristiku 3D modelu, čo zvyšuje stabilitu v AR aplikáciách.

5 Špecifikácia požiadaviek systému

Práca predstavuje moderný prístup k priemyselnému internetu vecí (Industrial Internet of Things (slovensky „priemyselný internet vecí“) (IIoT)), kde mobilná aplikácia slúži ako primárne rozhranie pre obsluhu výrobných linky, prepojené s riadiacim systémom prostredníctvom edge computing zariadenia umiestneného priamo v prevádzke.

5.1 Prehľad systému

V moderných priemyselných prevádzkach narastá tlak na flexibilitu a mobilitu obsluhy výrobných zariadení. Tradičné riešenia sú spravidla viazané na pevne umiestnené operátorské panely alebo stolové počítače, čo obmedzuje pohyblivosť obsluhy a zvyšuje reakčný čas pri riešení prevádzkových udalostí. Mobilné zariadenia ponúkajú možnosť prístupu k výrobným systémom odkiaľkoľvek v rámci areálu závodu. Ich použitie v kombinácii s jednoznačnou identifikáciou zariadení otvára cestu k bezpečnému a intuitívnemu ovládaniu výrobných linky.

Kľúčovou výzvou pri takomto prístupe je spoľahlivé a jednoznačné spárovanie mobilného zariadenia s konkrétnou fyzickou časťou výrobných linky. Skenovaním vizuálneho identifikátora umiestneného priamo pri zariadení operátor okamžite nadviaže spojenie a systém presne vie, kto a ktoré zariadenie ovláda. Toto spojenie je základom pre bezpečné riadenie, auditovanie operácií a správu prístupu viacerých používateľov k rovnakému zariadeniu.

Ďalšou dôležitou oblasťou je správa prístupu pracovníkov s rôznymi rolami. Operátor potrebuje priamy prístup k riadeniu linky, manažér vyžaduje prehľad o výkonnosti a vyťaženosť, údržbár musí mať garantovaný exkluzívny, neprepisovateľný prístup pri servisnom zásahu a administrátor spravuje celý systém vrátane registrácie nových používateľov. Bez jasne definovaného mechanizmu riadenia prístupu by mohlo dôjsť ku konfliktom príkazov alebo neoprávnenému zásahu do výrobného procesu.

Celá komunikačná vrstva od PLC až po mobilnú aplikáciu prebieha cez štandardné priemyselné a webové protokoly. Backendová infraštruktúra je plne kontajnerizovaná a to umožňuje rýchle nasadenie na ďalšie uzly v rámci továrne. Ovládanie linky je obmedzené výhradne na lokálnu sieť továrne a cloudová vrstva slúži len na vzdialený reporting a zálohovanie agregovaných dát.

Neoddeliteľnou súčasťou systému je inteligentný technický asistent. Asistent odpovedá na technické otázky obsluhy výhradne na základe dokumentácie a manuálov nahratých pre konkrétne zariadenie, čím znižuje závislosť od papierových príručiek a skracaje čas potrebný na riešenie prevádzkových problémov.

5.2 Popis používateľských rolí

Systém implementuje riadenie prístupu založené na rolách (*Role-Based Access Control*, RBAC). Každý nový používateľ sa musí zaregistrovať a pred prvým prihlásením čaká na schválenie administrátorom, ktorý mu zároveň priradí príslušnú rolu. Sú definované štyri roly:

Operátor: Základná obsluha výrobnéj linky. Prihlasuje sa na konkrétnu linku skenovaním ArUco kódu, odosiela riadiace príkazy, upravuje prevádzkové parametre a sleduje aktuálny stav zariadenia v reálnom čase. Nemá prístup k reportom, správe vedomostnej bázy ani správe používateľov. Môže v prípade potreby využiť AI asistenta.

Manažér: Zameriava sa na sledovanie výkonnostných ukazovateľov. Má prístup k histórii prihlásení jednotlivých operátorov, dennému reportu o vyťaženosti a agregovaným štatistikám pre všetky zariadenia. Linku priamo neovláda.

Údržbár: Má exkluzívnu ochranu relácie a žiadny iný používateľ ho nemôže odhlásiť z prihláseného zariadenia, čo zaručuje bezpečnosť počas servisného zásahu. Spravuje vedomostnú bázu (KB) pre jednotlivé zariadenia, pričom Knowledge Base (slovensky „znalostná báza“) (KB) tvorí znalostný zdroj pre AI asistenta.

Administrátor: Spravuje celý systém, schvaľuje nové registrácie, priraduje roly, edituje a odstraňuje používateľské kontá (s výnimkou hlavného admina). Má prístup ku všetkým logom a taktiež spravuje vedomostnú bázu.

5.3 Use-case scenáre

ID	UC-01: Registrácia a prihlásenie
Aktér	Všetky roly
Popis	Používateľ sa zaregistruje zadaním údajov. Kým administrátor neschváli konto, aplikácia zobrazí informáciu o čakaní na schválenie. Po schválení sa používateľ prihlási a získa JWT token.

ID	UC-02: Skenovanie ArUco kódu a spárovanie linky
Aktér	Operátor, údržbár
Popis	Používateľ naskenuje ArUco kód pri zariadení. Ak je linka voľná, po potvrdení dialógu sa vytvorí relácia. Ak je obsadená, zobrazí sa identita a rola prihláseného. Reláciu neprítomného operátora je možné prevziať; reláciu údržbára prevziať nie je povolené.

ID	UC-03: Ovládanie linky v reálnom čase
Aktér	Operátor
Popis	Operátor odosiela príkazy z ovládacieho panelu. Každý príkaz má unikátny identifikátor a prechádza stavmi: <i>queued</i> → <i>sent</i> → <i>acknowledged</i> → <i>applied</i> (resp. <i>failed</i> / <i>timeout</i>). Pri <i>timeout</i> aplikácia upozorní používateľa s možnosťou opakovania príkazu.
ID	UC-04: Monitorovanie stavu linky
Aktér	Operátor, manažér
Popis	Dashboard zobrazuje procesné dáta z PLC v reálnom čase cez WebSocket. Stavý sú farebne kódované podľa normy IEC 60073.
ID	UC-05: Prezeranie logov a reportov
Aktér	Manažér, admin
Popis	Po výbere zariadenia zo zoznamu sa zobrazí história prihlásení, denný report o vyťažnosti operátorov a prehľad udalostí.
ID	UC-06: Správa vedomostnej bázy
Aktér	Údržbár, admin
Popis	Prostredníctvom CRUD rozhrania sa nahrávajú, upravujú a odstraňujú dokumenty KB (texty a obrázky) pre každé zariadenie.
ID	UC-07: AI asistent (RAG chatbot)
Aktér	Všetky roly
Popis	Tlačidlom v pravom hornom rohu každej obrazovky (okrem prihlasovacej) sa spustí chatbot. Odpovedá výhradne na základe KB daného zariadenia; ak informácia chýba, prizná to. Rozhranie aj odpovede sú lokalizované v SK a EN.
ID	UC-08: Správa používateľov
Aktér	Admin
Popis	Admin schvaľuje registrácie, priraduje roly, upravuje a odstraňuje kontá. Hlavného admina nie je možné odstrániť ani upraviť inou cestou.
ID	UC-09: Nastavenia aplikácie
Aktér	Všetky roly
Popis	V nastaveniach je možné zmeniť jazyk (SK / EN) a vlastné heslo. Jazyk je dostupný aj na prihlasovacej obrazovke.

ID	UC-10: Cloudová synchronizácia
Aktér	Systém (automaticky)
Popis	Agregované dáta sú periodicky synchronizované do cloudu. Vzdialený prístup slúži výhradne na čítanie reportov a riadiace príkazy sú povolené iba v lokálnej sieti závodu.

5.4 Funkcionálne požiadavky

- FP1:** Aplikácia musí umožniť načítanie unikátneho identifikátora na automatické spárovanie mobilného zariadenia s konkrétnou časťou výrobnéj linky.
- FP2:** Používateľ musí mať možnosť odosielať riadiace príkazy a modifikovať prevádzkové parametre v riadenom systéme.
- FP3:** Systém musí implementovať verifikáciu kritických príkazov. Po odoslaní požiadavky musí aplikácia čakať na potvrdenie z PLC a vizuálne indikovať úspešné vykonanie zmeny.
- FP4:** Zobrazovanie aktuálnych stavov z PLC.
- FP5:** Implementácia chatbota využívajúceho metódu *Retrieval-Augmented Generation*. Chatbot musí odpovedať na technické otázky výhradne na základe poskytnutej vedomostnej bázy.
- FP6:** Plná lokalizácia používateľského rozhrania a odpovedí chatbota v dvoch jazykoch: slovenský (SK) a anglický (EN).
- FP7:** Agregované štatistické dáta musia byť periodicky zálohované z lokálnej databázy do cloudu pre potreby vzdialeného prístupu.
- FP8:** Aplikácia musí implementovať mechanizmus exkluzívnej relácie pre ovládanie konkrétnej výrobnéj linky. Po úspešnom prihlásení a spárovaní linky sa vytvorí aktívna relácia viazaná na používateľa. Relácia musí byť udržiavaná periodickým *heartbeat* signálom a automaticky ukončená pri odhlásení alebo po prekročení časového limitu neaktivity.
- FP9:** Pri pokuse o spárovanie linky, ktorá je už obsadená aktívnou reláciou, musí aplikácia používateľovi zobraziť informáciu o existujúcej relácii (identifikátor používateľa/rola). Nútené ukončenie resp. prevzatie relácie musí byť umožnené v prípade neodhlásenia sa už neprítomnej obsluhy.

FP10: Systém musí podporovať prihlásenie používateľov pomocou autentifikácie založenej na JWT. Aplikácia musí implementovať *Role-Based Access Control* (RBAC) s rolami: operátor, manažér, údržbár, admin.

FP11: Každý odoslaný riadiaci príkaz musí mať unikátny identifikátor (*command_id*) a systém musí poskytovať stav príkazu minimálne v úrovniach: *queued*, *sent*, *acknowledged*, *applied*, *failed*, *timeout*. Aplikácia musí tieto stavy používateľovi vizualizovať.

FP12: Systém musí ukladať základný log operácií obsahujúci minimálne: čas vykonania, identifikátor používateľa, identifikátor linky, typ akcie/príkazu a výsledok vykonania. Log musí byť dostupný pre roly *manažér* a *admin*.

5.5 Nefunkcionálne požiadavky

Definujú kvalitatívne atribúty, technologické obmedzenia a nároky na bezpečnosť.

NFP1: Celá backendová infraštruktúra musí byť kontajnerizovaná pomocou nástroja Docker a spustená na edge zariadení.

NFP2: Prenos správ v edge infraštruktúre musí prebiehať cez MQTT s QoS 1 alebo QoS 2.

NFP3: Latencia riadiaceho cyklu nesmie presiahnuť 250 ms.

NFP4: Odpovede AI asistenta musia byť striktne viazané na poskytnutý kontext. Ak informácia v materiáloch chýba, model musí túto skutočnosť priznať.

NFP5: Lokálna databáza musí byť konfigurovaná na zachovanie integrity dát aj v prípade náhleho prerušenia napájania edge zariadenia.

NFP6: Využitie Docker Compose musí umožňovať rýchle nasadenie identického stacku na iné edge uzly v rámci továrne.

NFP7: Systém musí zabrániť viacnásobnému vykonaniu toho istého príkazu v prípade opakovaného doručenia správy. Každý príkaz musí obsahovať unikátny identifikátor a backend musí zabezpečiť, že príkaz s rovnakým identifikátorom bude spracovaný najviac raz.

NFP8: Pri nedoručení potvrdenia (ACK) do 1.5 s musí aplikácia označiť príkaz ako *timeout* a informovať používateľa o neúspechu vykonania s možnosťou opakovania požiadavky. Systém musí logovať dôvod zlyhania, ak je dostupný.

NFP9: Ovládacie príkazy z externých sietí mimo lokálnej siete závodu nesmú byť povolené. Vzdialený prístup (mimo LAN) môže byť použitý výhradne na prehliadanie agregovaných dát a reportov v cloude.

NFP10: Backendové služby musia poskytovať štruktúrované logy a základné prevádzkové metriky (napr. latencia riadiaceho cyklu, počet neúspešných príkazov, stav pripojenia k PLC) pre účely diagnostiky.

5.6 Doménové požiadavky

Požiadavky plynúce z priemyselného prostredia a použitých technológií.

DP1: Komunikácia s hardvérom PLC musí prebiehať cez natívny protokol S7.

DP2: Grafické rozhranie musí dodržiavať priemyselné štandardy pre farebnú indikáciu stavov podľa normy IEC 60073.

DP3: Systém musí pracovať v hybridnom režime – plná funkčnosť ovládania v lokálnej sieti aj pri výpadku pripojenia k internetu.

6 Voľba technológií

Výber technologického stacku bol výsledkom posúdenia viacerých faktorov: požiadaviek konkrétneho hardvéru v prevádzke, charakteru komunikačných tokov, integračných potrieb AI vrstvy a praktických skúseností vývojového tímu. Táto kapitola zdôvodňuje každé kľúčové rozhodnutie.

6.1 Komunikácia s PLC – S7 protokol

Riadiaci systém v prevádzke tvorí PLC Siemens S7-1200, pre ktorý je natívnym komunikačným protokolom S7. Práve táto skutočnosť bola primárnym dôvodom jeho výberu – S7 protokol umožňuje priame čítanie a zápis dátových blokov, merkerov a vstupno-výstupných obrazov bez nutnosti konfigurovať ďalšiu middleware vrstvu priamo na PLC.

Alternatívou by bolo aktivovanie servera OPC UA priamo v S7-1200, čo je síce technicky možné, avšak prináša niekoľko nevýhod: vyšší nápor na výpočtové zdroje PLC s obmedzenou pamäťou, potrebu licencie v niektorých konfiguráciách a výrazne komplexnejší konfiguračný proces. S7 protokol cez knižnicu `node-red-contrib-s7` poskytuje priamy a odlahčený prístup k dátam PLC bez týchto obmedzení.

- **Výhody:** Natívna podpora pre Siemens S7-1200, priamy prístup k dátovým blokom, nízka réžia, jednoduchá konfigurácia v Node-RED.
- **Nevýhody:** Viazanosť na platformu Siemens, absencia štandardizovaného informačného modelu (oproti OPC UA).

6.2 Komunikačná middleware vrstva – Node-RED

Medzi PLC a zvyškom systému stojí Node-RED – vizuálny, tokovým programovaním riadený nástroj bežiaci ako Docker kontajner na edge zariadení v prevádzke. Node-RED periodicky číta hodnoty z PLC prostredníctvom S7 protokolu a publikuje ich do MQTT brokera, čím premostuje svet OT (PLC, S7) a IT (MQTT, backend).

Táto vrstva bola zvolená z niekoľkých dôvodov. Disponuje natívnym uzlom `node-red-contrib-s7` pre priamu komunikáciu so Siemens PLC a rovnako natívnou podporou MQTT. Toková vizuálna paradigma umožňuje definovať, transformovať a smerovať dátové toky bez potreby písať rozsiahly kód, čo výrazne skracuje čas konfigurácie. Node-RED tiež dobre zvláda periodické dotazovanie (polling) PLC a podmienené spúšťanie akcií.

- **Výhody:** Vizuálne toky, natívna S7 a MQTT podpora, jednoduchý deployment v Dockeri, aktívna komunita s rozsiahlou knižnicou uzlov.

- **Nevýhody:** Nie je vhodný pre komplexnú biznis logiku – tú je lepšie presunúť do backendovej vrstvy; vizuálne toky môžu byť pri väčšom rozsahu ťažšie udržiavateľné.

6.3 Transportný protokol – MQTT namiesto OPC UA

Pre transport dát medzi edge vrstvou a backendom bol zvolený protokol MQTT. Kľúčovým argumentom je jeho nenáročnosť – protokol bol navrhnutý pre zariadenia s obmedzenými zdrojmi a nestabilné siete, čo presne zodpovedá podmienkam edge zariadenia v priemyselnej prevádzke.

OPC UA je komplexný štandard s bohatým informačným modelom, vstavanou bezpečnosťou a sémantickým popisom dát. Pre scenár, kde Node-RED číta hodnoty zo Siemens PLC a odovzdáva ich ďalej na spracovanie, táto komplexnosť nepredstavuje pridanú hodnotu, ale zbytočnú záťaž. MQTT broker (Mosquitto) nasadený v Dockeri zvládne rovnakú úlohu s minimálnou pamäťovou a CPU réžiou, pričom model publish-subscribe prirodzene zodpovedá reaktívnemu charakteru priemyselných dát – backend odberá len témy, ktoré ho zaujímajú.

- **Výhody MQTT:** Nízka réžia, jednoduchá architektúra pub/sub, výborná podpora v Node-RED aj Pythone, QoS úrovne pre spoľahlivosť doručenia, voľne dostupný broker Mosquitto.
- **Kedy by bol OPC UA lepšou voľbou:** Pri požiadavke na štandardizovaný sémantický popis dát, pri integrácii viacerých heterogénnych zariadení od rôznych výrobcov alebo pri prísnych bezpečnostných požiadavkách na úrovni transportnej vrstvy.

6.4 Backendový framework – FastAPI

Backend systému PLEXO bol implementovaný v Pythone s využitím frameworku FastAPI. Rozhodnutie pre Python bolo determinované predovšetkým požiadavkou na integráciu RAG pipeline – knižnice LangChain, ChromaDB a Ollama sú súčasťou pythonovského ekosystému a ich integrácia do toho istého procesu eliminuje potrebu medziprocesovej komunikácie.

FastAPI bol zvolený spomedzi pythonovských frameworkov z viacerých dôvodov. Asynchrónna architektúra (Asynchronous Server Gateway Interface (slovensky „asynchrónne rozhranie serverovej brány“) (ASGI)) umožňuje efektívne obsluhovať súbežné požiadavky mobilnej aplikácie bez blokovania počas čakania na odpoveď jazykového modelu. Automatické generovanie interaktívnej OpenAPI dokumentácie (Swagger User

Interface (slovensky „používateľské rozhranie“) (UI)) výrazne uľahčuje vývoj a testovanie API. Validácia vstupných dát cez Pydantic znižuje množstvo boilerplate kódu. V poslednom rade hrali rolu aj existujúce skúsenosti s týmto frameworkom, čo priamo skracuje čas vývoja.

- **Výhody:** Asynchrónne spracovanie požiadaviek, automatická OpenAPI dokumentácia, Pydantic validácia, priama integrácia s AI knižnicami, rýchly vývoj.
- **Nevýhody:** Interpretovaný Python je pomalší oproti kompilovaným jazykom pri CPU-intenzívnych operáciách; pre čistý REST API bez AI vrstvy by bol Go výkonnejší.

6.5 Mobilný frontend – Flutter

Pre mobilnú aplikáciu bol zvolený framework Flutter od spoločnosti Google. Primárnou cieľovou platformou je Android, avšak Flutter umožňuje z jednej kódovej základne generovať aj webovú aplikáciu, čo otvára možnosť implementácie administrátorského panelu bez nutnosti udržiavať samostatný projekt.

Oproti natívnemu vývoju v Kotlin Flutter ponúka výrazne rýchlejší vývojový cyklus vďaka funkciám hot reload a hot restart. Vlastný vykresľovací engine (Impeller) zaručuje vizuálnu konzistenciu rozhrania naprieč zariadeniami a operačnými systémami. Bohatá štandardná knižnica widgetov pokrýva väčšinu potrieb priemyselného ovládacieho rozhrania vrátane dynamických grafov, stavových indikátorov a formulárov.

- **Výhody:** Jedna kódová základňa pre Android aj web, rýchly vývojový cyklus, konzistentné UI, silná podpora od Google a rastúca komunita.
- **Nevýhody:** Väčšia veľkosť výslednej aplikácie oproti natívnym riešeniam; pri požiadavke na plnohodnotnú iOS aplikáciu by bolo potrebné doplniť testovanie na Apple zariadeniach.

6.6 TODO: Identifikátor - ArUco

Preco aruco - vedia citat AGV resp upratovací roboti Ake experimetny sme robili s 3d identifikatormi

7 Experimentálne porovnanie a výber jazykových modelov

7.1 Popis testovaných modelov

Na účely porovnania bolo vybraných päť lokálne spustiteľných jazykových modelov dostupných prostredníctvom platformy Ollama. Modely boli zámerne zvolené tak, aby pokrývali tri veľkostné kategórie: modely do 10 miliárd parametrov, modely v rozsahu 10 až 15 miliárd parametrov a model s 24 miliardami parametrov. Cieľom tohto rozloženia bolo analyzovať vzťah medzi veľkosťou modelu, kvalitou odpovedí a časom odozvy.

1. llama3.2:3b

Kompaktný model od spoločnosti Meta s 3 miliardami parametrov zo série Llama 3.2, ktorá zahŕňa predtrénované aj inštrukčne doladené generatívne modely vo veľkostiach 1B a 3B parametrov. Inštrukčne doladené varianty sú optimalizované pre multijazyčné konverzačné úlohy vrátane agentického vyhľadávania a sumarizácie. Podľa oficiálnej dokumentácie model prekonáva modely Gemma 2 2.6B a Phi 3.5-mini v úlohách ako sledovanie inštrukcií, sumarizácia, prepis promptov a používanie nástrojov [ollama_llama32]. Do testovania bol zaradený ako zástupca najmensej veľkostnej kategórie s predpokladom najrýchlejšej odozvy.

2. gemma4:e4b

Model od spoločnosti Google zo série Gemma 4 s 4 miliardami parametrov v kvantizovanej verzii (e4b označuje efektívnu 4-bitovú kvantizáciu). Série Gemma 4 predstavuje najnovšiu generáciu otvorených modelov Google, navrhnutú s dôrazom na multimodálne schopnosti a efektívne nasadenie na bežnom spotrebiteľskom hardvéri. Model podporuje dlhé kontextové okno a je optimalizovaný pre inštrukčné úlohy vrátane multijazyčných konverzačných scenárov [ollama_gemma4]. Do testovania bol zaradený ako zástupca strednej veľkostnej kategórie s priaznivým pomerom výkonu a pamätevej náročnosti.

3. mistral-nemo:12b

Model s 12 miliardami parametrov vyvinutý spoločnosťami Mistral AI a NVIDIA. Disponuje kontextovým oknom až 128 000 tokenov a podľa oficiálnej dokumentácie dosahuje v svojej veľkostnej kategórii špičkové výsledky v oblasti uvažovania, znalosti sveta a presnosti generovania kódu. Vďaka štandardnej architektúre je plnohodnotnou náhradou modelu Mistral 7B v existujúcich systémoch [ollama_mistralnemo]. Do testovania bol zaradený ako zástupca kategórie 10–20 miliárd parametrov.

4. **phi4:14b**

Model od spoločnosti Microsoft s 14 miliardami parametrov a kontextovým oknom 16 000 tokenov. Je trénovaný na kombinácii syntetických dátových sád, filtrovaných verejne dostupných webových zdrojov a akademických kníh a dátových sád otázok a odpovedí. Prešiel procesom doladenia pomocou supervised fine-tuningu a direct preference optimization s dôrazom na presné sledovanie inštrukcií a bezpečnosť. Podľa oficiálnej dokumentácie je určený pre prostredia s obmedzenými výpočtovými zdrojmi, scenáre s požiadavkami na nízku latenciu a úlohy vyžadujúce uvažovanie a logiku [ollama_phi4]. Do testovania bol zaradený ako druhý zástupca kategórie 10–20 miliárd parametrov.

5. **mistral-small:24b**

Model od spoločnosti Mistral AI s 24 miliardami parametrov a kontextovým oknom 32 000 tokenov, ktorý stanovuje nový benchmark v kategórii modelov do 70B parametrov. Podľa oficiálnej dokumentácie dosahuje výkon porovnateľný s modelmi trojnásobnej veľkosti a proprietárnym modelom GPT-4o mini naprieč úlohami v oblasti kódu, matematiky, všeobecných znalostí a sledovania inštrukcií [ollama_mistralsmall]. Model vyniká multijazyčnou podporou, natívnym volaním funkcií, JavaScript Object Notation (slovensky „JavaScriptová objektová notácia“) (JSON) výstupom a silnou adherenciou k systémovým promptom. Vďaka kompaktnej veľkosti je nasaditeľný lokálne na bežnom spotrebiteľskom hardvéri a je licencovaný pod Apache 2.0. V rámci projektu PLEXO bol tento model používaný ako primárny model asistenta pred spustením porovnávacieho testovania a v tomto porovnaní slúži ako referenčná hodnota maximálnej kvality lokálne spustiteľných riešení.

7.2 Metodika testovania

Testovanie prebiehalo formou automatizovaného benchmarku implementovaného v jazyku Python s využitím knižníc LangChain, ChromaDB a Ollama. Znalostná báza asistenta PLEXO vo formáte `.txt` bola rozdelená na dokumentové sekcie podľa markerov `-- DOKUMENT: --` a uložená do vektorovej databázy ChromaDB s využitím embedding modelu `nomic-embed-text`. Pre každú testovú otázku boli z vektorovej databázy vytiahnuté tri najrelevantnejšie dokumenty, ktoré boli spolu s otázkou odovzdané modelu ako kontext.

Každý model bol otestovaný na sade 14 testovacích otázok rozdelených do troch kategórií:

- **Faktické otázky** (10 otázok) – odpoveď je priamo obsiahnutá v znalostnej báze. 6 otázok v slovenskom a 4 otázky v anglickom jazyku.
- **Syntézne otázky** (2 otázky) – správna odpoveď vyžaduje kombináciu informácií z viacerých dokumentov znalostnej bázy.
- **Otázky mimo rozsahu** (2 otázky) – otázky, ktorých odpoveď sa v znalostnej báze nenachádza a model by mal odpovedať odmietnutím a odkazom na technika.

Na rozdiel od automatických metrík ako ROUGE, ktoré vykazujú nízku spoľahlivosť pri skloňovaných jazykoch ako slovenčina, bolo hodnotenie kvality realizované manuálne. Každá odpoveď bola ohodnotená podľa nasledujúcich kritérií:

- **Relevancia** (škála 1–5) – miera, do akej odpoveď zodpovedá položenej otázke.
- **Úplnosť** (škála 1–5) – pokrytie všetkých dôležitých informačných bodov správnej odpovede.
- **Zrozumiteľnosť** (škála 1–5) – jasnosť a zrozumiteľnosť formulácie odpovede.
- **Jazyková správnosť** (škála 1–3) – či model odpovedal v rovnakom jazyku, v akom bola položená otázka.
- **Odmietnutie otázky mimo rozsahu** (áno/nie) – či model správne odmietol odpovedať na otázku, ktorá nie je pokrytá znalostnou bázou.

Celkové skóre bolo vypočítané ako vážený priemer jednotlivých kritérií: relevancia (30 %), úplnosť (25 %), zrozumiteľnosť (20 %) a jazyková správnosť (25 %).

Okrem kvalitatívnych metrík bol pre každú odpoveď zaznamenaný čas odozvy v milisekundách. Testovanie prebiehalo na notebookovom hardvéri (CPU AMD Ryzen 5 5600H, RAM 32 GB, GPU NVIDIA RTX 3060 6 GB VRAM). Keďže väčšina testovaných modelov presahuje kapacitu VRAM, modely bežali čiastočne alebo úplne na systémovej RAM, čo predstavuje hardvérovú limitáciu meraní. V produkčnom nasadení na dedikovanom serveri s dostatočnou VRAM by časy odozvy boli výrazne nižšie.

7.3 Výsledky porovnania

Výsledky hodnotenia všetkých piatich modelov sú zhrnuté v tabuľke 3.

Tabuľka 3: Výsledky porovnania jazykových modelov

Model	Priem. čas [ms]	Relev. (1–5)	Úplnosť (1–5)	Zrozum. (1–5)	Jazyk (1–3)	OOS odmiet.	Celk. skóre
llama3.2:3b	4 536	2,29	2,21	2,50	2,14	100 %	2,63
gemma4:e4b	11 165	4,86	4,86	5,00	3,00	100 %	4,92
mistral-nemo:12b	15 217	4,07	4,07	4,29	2,86	100 %	4,29
phi4:14b	20 039	2,00	2,00	2,07	2,79	100 %	2,68
mistral-small:24b	35 801	1,86	1,86	1,86	3,00	100 %	2,65

Tabuľka 4: Priemerná relevancia podľa typu otázky a jazyka

Model	Faktická otázka	Syntéza	Mimo rozsahu	SK	EN
llama3.2:3b	1,70	2,50	5,00	2,12	2,50
gemma4:e4b	5,00	4,00	5,00	4,88	4,83
mistral-nemo:12b	4,20	2,50	5,00	4,62	3,33
phi4:14b	1,40	2,00	5,00	2,00	2,00
mistral-small:24b	1,40	1,00	5,00	2,00	1,67

7.4 Analýza výsledkov a výber modelu

Z výsledkov testovania vyplýva, že model **gemma4:e4b** dosiahol najvyššie celkové skóre 4,92/5,0 spomedzi všetkých testovaných modelov. Model vykazoval konzistentne vysokú kvalitu odpovedí naprieč všetkými typmi otázok aj oboma testovacími jazykmi, pričom jeho priemerný čas odozvy 11 165 ms bol v podmienkach notebookového hardvéru prijateľný a takmer trojnásobne nižší v porovnaní s modelom mistral-small:24b.

Pozoruhodným zistením je skutočnosť, že modely phi4:14b a mistral-small:24b opakovane odmietali odpovedať na otázky, ktorých odpovede boli v znalostnej báze jednoznačne obsiahnuté. Tento jav naznačuje, že RAG pipeline nepodal týmto modelom dostatočne relevantný kontext, čo mohlo byť spôsobené odlišnou citlivosťou modelov na formát vstupného kontextu alebo prísnejším nastavením ich systémového promptu.

Všetky testované modely dosiahli 100 % úspešnosť pri odmietaní otázok mimo rozsahu znalostnej bázy, čo potvrdzuje správne fungovanie systémového promptu z hľadiska bezpečnosti systému.

7.4.1 Overenie na cloudovom hardvéri

V nadväznosti na merania realizované lokálne bolo vybrané testovanie vykonaných aj na platforme RunPod.io, ktorá poskytuje prístup k výkonným GPU serverom formou platby za využitý čas. Cieľom bolo overiť, či hardvérová limitácia notebookového hardvéru skresľuje závery o reálnej použiteľnosti modelov v produkcii.

Výsledky potvrdili predpoklad: na dedikovanom serverovom hardvéri s dostatočnou VRAM klesol priemerný čas odozvy na menej ako 8 sekúnd, čo je pre interaktívne použitie výrazne prijateľnejšia hodnota. Kvalita odpovedí pritom zostala nezmenená, čo je v súlade s očakávaním – model generuje rovnaký výstup nezávisle od rýchlosti hardvéru, rozdiel sa prejaví iba v latencii.

7.4.2 Záverečné rozhodnutie o architektúre AI asistenta

Napriek tomu, že výsledky testovania lokálnych modelov boli z kvalitatívneho hľadiska uspokojivé, cloudové overenie odhalilo zásadné ekonomické obmedzenie tohto prístupu. Modely, ktoré dosahujú dostatočnú kvalitu odpovedí (kategória 9–24B parametrov), si pre produkčné nasadenie vyžadujú dedikovaný GPU server s dostatočnou VRAM. Prevádzkové náklady takéhoto riešenia nie sú pre projekt tejto škály rentabilné.

Na druhej strane, menšie lokálne modely (do 4B parametrov), ktoré by bolo možné prevádzkovať na bežnom edge hardvéri bez výrazných nákladov, nedosahujú požadovanú kvalitu odpovedí pre technickú asistenciu v priemyselnom prostredí, kde je faktická presnosť kritická.

Z týchto dôvodov bolo rozhodnuté opustiť prístup lokálnych LLM a prejsť na volanie externého API – konkrétne Claude API od spoločnosti Anthropic. Tento prístup zabezpečuje prístup k výkonným jazykovým modelom bez nutnosti prevádzkovať vlastný serverový hardvér, pričom náklady sa platia priamo úmerne skutočnému využitiu (model per-token). RAG pipeline a znalostná báza zostávajú zachované v pôvodnej podobe – kontextové dokumenty sú naďalej vyhľadávané lokálne z vektorovej databázy a odovzdávané modelu ako súčasť promptu. Mení sa iba vrstva generovania odpovede, ktorá prechádza z lokálne bežiaceho modelu na cloudové API.

8 TOOD: Aplikácia a jej funkcionality

8.1 Skener Aruco kodov

Dostupné pre: operator, technik, manager, admin Slúži pre prihlásenie sa na zariadenie pomocou naskenovania Aruco kodu umiestneného na zariadení.

8.2 Generátor Aruco kódov

Dostupné pro: technik, manager, admin Pri zakladaní nového zariadenia systém vygeneruje unikátny Aruco kód, ktorý sa vytlačí a umiestni na zariadenie. Generátor Aruco kódov dokáže generovať kódy vo formáte .png, ale aj modely týchto kódov v .stl, kde biela časť kódu je vystúpená a čierna časť je zapustená.

8.3 Monitoring a ovládanie zariadení

Dostupné pro: operator, technik, admin Hlavná funkcionality aplikácie, ktorá umožňuje monitorovať aktuálny stav zariadení (napr. teplotu, tlak, stav pohybu) a ovládať základné funkcie (napr. spustenie, zastavenie, reset) pomocou jednoduchého a intuitívneho rozhrania.

8.4 Virtuálny asistent PLEXO

Dostupné pro: operator, technik, manager, admin Virtuálny asistent PLEXO poskytuje používateľom možnosť klásť otázky týkajúce sa zariadení, ich stavu, postupov údržby a riešenia problémov. Asistent využíva integrovaný jazykový model na poskytovanie relevantných a presných odpovedí založených na znalostnej báze systému.

8.5 Správa používateľov

Dostupné pro: manager, admin Správa používateľov umožňuje manažérom a administrátorom vytvárať, upravovať a mazať používateľské účty, pridávať role a resetovať heslá. Táto funkcionality zabezpečuje, že každý používateľ má prístup len k tým častiam systému, ktoré sú relevantné pre jeho rolu.

8.6 Správa zariadení

Dostupné pro: technik, manager, admin Keďže zariadenia môžu časom pribúdať alebo sa mierne zmeniť, je dôležité mať možnosť ich spravovať. Táto funkcionality umožňuje technikom a manažérom pridávať nové zariadenia do systému, upravovať ich konfiguráciu

(napr. nastavenie parametrov monitoringu) a odstraňovať nefunkčné alebo nepotrebné zariadenia.

8.7 Správa Knowledgebase

Dostupné pro: technik, manager, admin Správa znalostnej bázy umožňuje technikom a manažérom aktualizovať a rozširovať obsah, na ktorom je založený virtuálny asistent PLE XO. Táto funkcion alita zahŕňa prídávanie nových dokumentov, aktualizáciu existujúcich informácií a organizáciu znalostnej bázy pro optimalizáciu relevantnosti odpovedí asistenta.

8.8 Nastavenia

Dostupné pro: operator, technik, manager, admin nastavenie jazykovej preferencie, zmena hesla

Záver

„V tejto príručke sme sa snažili poskytnúť praktické pokyny pre vypracovanie záverečných prác na Fakulte elektrotechniky a informatiky STU v Bratislave. Obsahuje dôležité informácie o formálnej stránke dokumentu, ktoré sú nevyhnutné pre zabezpečenie jasnej štruktúry a správneho formátovania práce. Naša príručka kladie dôraz na dodržiavanie základných pravidiel, ktoré pomáhajú minimalizovať chyby a zlepšujú celkovú kvalitu predkladaných dokumentov.

Rovnakú pozornosť venujeme aspektom, ako sú formátovanie textu, zarovnávanie, riadkovanie a použitie preddefinovaných štýlov, ktoré prispievajú k profesionálnemu vzhľadu dokumentu. Dôkladné spracovanie matematických rovníc a grafov je ďalším dôležitým prvkom, ktorý môže ovplyvniť hodnotenie práce. Práca s týmito aspektmi zaisťuje, že obsah je prehľadný a presne odráža zamýšľané myšlienky.

V neposlednom rade sa naša príručka zameriava na význam citovania externých zdrojov, čo je zásadné pre zachovanie integrity a etických štandardov pri písaní akademických textov. Správne citovanie nielenže posilňuje argumentáciu, ale tiež pomáha predchádzať plagiátorstvu, ktoré môže mať vážne následky na akademickú kariéru.

Dodržiavanie týchto pokynov umožňuje vyhnúť sa bežným chybám a zvyšuje kvalitu záverečných prác, čím sa zlepšuje celkový dojem z akademických schopností. Dobre vypracovaná práca môže viesť k pozitívnemu hodnoteniu a uznaniu zo strany hodnotiteľov, čo je cieľom každého študenta.

Celkovo je príručka navrhnutá ako cenný nástroj na podporu pri písaní záverečných prác. Poskytnuté informácie a odporúčania prispievajú k dosiahnutiu akademických cieľov a pomôžu vypracovať kvalitné a odborné dokumenty, ktoré budú spĺňať štandardy fakulty.“

(ChatGPT 3, 2024)

Poznámka autora: Časť záveru vygenerovala umelá inteligencia na základe analýzy obsahu verzie dokumentu pre práce písané v MS Word. Konkrétne išlo o aplikáciu ChatGPT 3 dňa 29. 9. 2024. Rukopis umelej inteligencie je z textu cítiť aj napriek tomu, že celá komunikácia vedúca k tejto forme záveru trvala približne 40 minút. Parafrázujúc ľudové príslovie konštatujeme, že umelá inteligencia je možno dobrý sluha, ale zlý pán. Používajme ju s rozumom.

Pripomíname, že takýto spôsob tvorby obsahu je v záverečnej práci, ako aj v iných akademických textoch, nepripustný. Autorské dielo nesmie obsahovať odseky vytvorené generatívnou umelou inteligenciou. Preto sme ho formátovali ako citovaný text.

Literatúra

1. RYBIČKA, J. *L^AT_EX pro začátečníky*. 3. vyd. Konvoj, 2003. ISBN 80-7302-049-1.

Použitie nástrojov umelej inteligencie

OpenAI (2025), ChatGPT 4o, časť ??, generovanie vzorca záznamu použitia AI.

OpenAI (2024), ChatGPT 3, Záver, generovanie textu.

OpenAI (2025), ChatGPT 4o, dodatok B, generovanie kódu programu.

OpenAI (2024), ChatGPT 3, dodatok C, generovanie zoznamu.

Dodatok A: Algoritmus

Algoritmus 1 Vypočítaj $y = x^n$

Require: $n \geq 0 \vee x \neq 0$

Ensure: $y = x^n$

$y \Leftarrow 1$

if $n < 0$ **then**

$X \Leftarrow 1/x$

$N \Leftarrow -n$

else

$X \Leftarrow x$

$N \Leftarrow n$

end if

while $N \neq 0$ **do**

if N is even **then**

$X \Leftarrow X \times X$

$N \Leftarrow N/2$

else $\{N \text{ is odd}\}$

$y \Leftarrow y \times X$

$N \Leftarrow N - 1$

end if

end while

Dodatok B: Výpis dlhého kódu

Dlhšie kódy, prípadne kompletne programy vypisujeme do dodatkov práce. Nasledujúci program v jazyku Python vygeneruje súbor `thesis.tex`, ktorý je funkčne totožný s hlavným súborom tohto dokumentu. Stačí vyplniť prehľadný formulár pri volaní funkcie `generate_thesis_tex` na konci kódu a získate hlavný súbor vašej práce.

```
def generate_thesis_tex(
    author,
    reg_nr,
    title,
    title_en,
    day,
    month,
    year,
    keywords,
    keywords_en,
    study_programme,
    study_programme_en,
    study_field,
    study_field_en,
    training_workplace,
    training_workplace_en,
    supervisor,
    consultant=None,
    thesis_type="bp",
    language="sk",
):

    tex_class_option = thesis_type if thesis_type in ["bp", "dp"] else "bp"
    tex_language_option = "en" if language == "en" else ""

    if tex_language_option:
        document_class = f"\\documentclass[{tex_class_option},{tex_language_option}]{FEIstyle}"
    else:
        document_class = f"\\documentclass[{tex_class_option}]{FEIstyle}"

    tex_content = f"""
{document_class}

% Author of the thesis
\\FEIauthor{{{author}}}
```

```

% Evidence number
\\FEIregNr{{{reg_nr}}}

% Title
\\FEItitle{{{title}}}
\\FEItitleEn{{{title_en}}}

% Date
\\FEIdate{{{day}}}{{{month}}}{{{year}}}

% Keywords
\\FEIkeywords{{{keywords}}}
\\FEIkeywordsEn{{{keywords_en}}}

% Further details
\\FEIstudyProgramme{{{study_programme}}}
\\FEIstudyProgrammeEn{{{study_programme_en}}}
\\FEIstudyField{{{study_field}}}
\\FEIstudyFieldEn{{{study_field_en}}}
\\FEItrainingWorkplace{{{training_workplace}}}
\\FEItrainingWorkplaceEn{{{training_workplace_en}}}

% Supervisor
\\FEISupervisor{{{supervisor}}}
""

    if consultant:
        tex_content += f"\n\\FEIconsultant{{{consultant}}}\n"

    tex_content += f""

% Bibliography database file
\\bibliography{{{includes/bibliography.bib}}}

\\begin{{document}}

\\frontmatter
\\FEIpdfInfo
\\FEIcover
\\FEItitlePage
\\includepdf[pages=-]{{{includes/assignment.pdf}}}
\\FEIthanks{{{includes/thanks}}}
\\FEIabstract{{{includes/abstract}}}
\\FEIabstractEn{{{includes/abstractEN}}}

```

```

\\FEIcontent
\\FEImanualListOfGlossaries{{includes/manual_glossary}}
\\FEIlistOfAlgorithms
\\FEIlistOfListings

\\mainmatter
\\FEIintroduction{{includes/introduction}}
\\FEIcore{{includes/core}}
\\FEIconclusion{{includes/conclusion}}
\\FEIbibliography
\\FEIaiDeclaration{{includes/ai_declaration}}
\\backmatter
\\FEIappendix{{Algoritmus\\label{{att:algorithms}}}}{{includes/attachmentA}}
\\FEIappendix{{Výpis dlhého kódu\\label{{att:listings}}}}{{includes/attachmentB}}
\\FEIappendix{{Slovníček pojmov\\label{{att:dictionary}}}}{{includes/attachmentC}}

\\end{{document}}
"""

with open("thesis.tex", "w", encoding="utf-8") as file:
    file.write(tex_content)

print("Súbor thesis.tex bol úspešne vygenerovaný.")

# Testovacie volanie funkcie
generate_thesis_tex(
    author="RNDr. Juraj Chlpík, PhD.",
    reg_nr="FEI-xxxx-xxxx",
    title="Rozšírená šablóna záverečnej práce na FEI STU v Bratislave v systéme LaTeX",
    title_en="Extended thesis template at FEI STU in Bratislava in LaTeX system",
    day="31",
    month="12",
    year="2024",
    keywords="záverečná práca, šablóna, LaTeX, formátovanie textu, citácie",
    keywords_en="Final thesis, template, LaTeX, text formatting, citations",
    study_programme="názov študijného programu",
    study_programme_en="Study program in English",
    study_field="názov študijného odboru",
    study_field_en="Field of the study in English",
    training_workplace="Názov školiaceho pracoviska",
    training_workplace_en="Training Work place",
    supervisor="tituly Meno Priezvisko, tituly",
    consultant="",

```

```
thesis_type="bp", # Možnosti: "bp", "dp"  
language="sk", # Možnosti: "sk" (slovenčina - predvolená), "en" (angličtina)  
)
```

Výpis kódu 1: Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra `\lstinputlisting`

Dodatok C: Slovníček pojmov

Abstrakt – Krátke zhrnutie hlavných bodov dokumentu, vrátane cieľov, metodológie a výsledkov.

Bod (pt) – Jednotka merania veľkosti písma; 1 bod sa rovná približne 0,352 mm.

Bold – Písmo, ktoré je tučné, používané na zvýraznenie dôležitých častí textu a v nadpisoch.

Citácia – Uvedenie zdroja informácií v texte, ktoré umožňuje identifikáciu pôvodného materiálu.

Font – Typ písma, ktorý sa používa na zobrazenie textu. Rôzne fonty majú odlišný štýl a vzhľad.

Graf – Vizualizácia údajov, ktorá zobrazuje vzťah medzi premennými.

Kapitola – Hlavná časť dokumentu, ktorá sa zaoberá konkrétnou témou a je zvyčajne označená číslom alebo názvom.

Korektúra – Proces revízie a opravy textu s cieľom odstrániť chyby.

Kurzíva – Typ písma, ktorý je sklonený doprava a používa sa na zvýraznenie textu, názvov alebo cudzích výrazov.

Metodológia – Opis metód a techník použitých pri výskume alebo analýze.

Normostrana – Základná jednotka pre meranie dĺžky textu, ktorá zvyčajne obsahuje 1 800 znakov vrátane medzier.

Obrázok – Grafický prvok alebo fotografia, ktorá ilustruje text.

Odsadenie – Vzdialenosť prvého riadku odseku od okraja stránky.

Odsek – Základná jednotka textu, ktorá obsahuje súvislé myšlienky. V $\text{\LaTeX}$ -u sa odseky oddeľujú prázdny riadkom v zdrojovom texte.

Plagiátorstvo – Prezenterovanie cudzej práce alebo myšlienok ako vlastných bez riadneho citovania.

Rez písma – Variant typu písma, ktorý určuje štýlové úpravy, ako sú normálny, tučný (bold), kurzíva (italic) alebo podčiarknutie.

Riadkovanie – Vzdialenosť medzi jednotlivými riadkami textu.

Sadzba – Proces usporiadania textu a grafiky na stránke.

Súbor – Elektronický dokument, ktorý obsahuje text alebo dáta.

Šablóna – Prednastavené formátovanie pre rôzne typy textu, ktoré zabezpečuje jednotný vzhľad.

Tabuľka – Usporiadanie údajov do riadkov a stĺpcov.

Titulný list – Prvá stránka dokumentu, ktorá obsahuje základné informácie, ako je názov práce a meno autora.

Typografia – Umenie a technika usporiadania textu tak, aby bol esteticky príťažlivý a čitateľný.

Úvod – Prvá časť dokumentu, ktorá predstavuje tému a ciele výskumu.

Vedecká práca – Akademický dokument, ktorý prezentuje výsledky výskumu, analýzy alebo diskusie o špecifickej téme.

Zdroje primárne a sekundárne – Primárne zdroje sú originálne dokumenty; sekundárne zdroje interpretujú alebo analyzujú primárne zdroje.